# Estimation Methods

Introduction To Estimation Methods

Stellenbosch University, Faculty of Medicine and Health Sciences,
Department of Global Health, Division of Biostatistics and Epidemiology,
Dr Innocent Maposa

April 2026

## Methods of Moments

### Introduction to Methods of Moments

- Matching of sample moment with population moments: Method of moments (MM)
- The Method of Moments is a simple technique based on the idea that the sample moments are "'natural"' estimators of population moments.
- For the first moment, we equate to sample mean!

$$E(X) = \mu = \left\{ \begin{array}{cc} \int_{-\infty}^{\infty} xf(x)dx & if \quad X \quad continuous \\ \sum_{\forall x} xP(X=x) & if \quad X \quad discrete \end{array} \right\}$$

  Then the MM method

$$\hat{\mu_1} = \frac{\sum_{i=1}^{n} X_i}{n}$$

- and the second moment:

$$\hat{\mu_2} = \frac{\sum_{i=1}^{n} X_i^2}{n}$$

### Method of Moments Example

- Let $X_1, X_2, \ldots, X_n$ be a sample from a Poisson distribution with parameter $\theta$. Find:
  1. The estimator of $\hat{\theta}$
  2. The estimator for variance of X, Var(X).
  3. Show that $\hat{\theta}$ is an unbiased estimator for $\theta$
- Let $X_1, X_2, \ldots, X_n$ be a sample from an $exp(\lambda)$. Use the method of moments to determine
  1. The estimator for $\hat{\lambda}$
  2. Estimator for the variance of X.
  3. Determine if $\hat{\lambda}$ is an unbiased and consistent estimator for $\lambda$.

# Maximum Likelihood Estimation

## Definition: Likelihood

- A likelihood for a statistical model is defined by the same formula as the density function, but with roles for data X and parameter $\theta$ interchanged.

$$L(\theta, X) = f(X, \theta)$$

- The likelihood is considered a function of $\theta$ for fixed data X, whereas the density is considered a function of X for fixed $\theta$. Both are the same function of X and $\theta$ jointly.
- Likelihood is actually a slightly more general concept

$$L(\theta, X) = h(X)f(X, \theta)$$

  which is a likelihood for the model when $h(X)$ is any strictly positive valued function of X that does not depend on or contain $\theta$ parameter.
- Reason for this generalization is that all of the uses we make of of the likelihood function will not be affected in any way by the presence or absence of $h(X)$.

## The ML estimation

The method of ML uses an estimator of the unknown true parameter value, the point $\hat{\theta_x}$ that maximizes the likelihood $L_x$.

- This estimator is called the Maximum Likelihood Estimator (MLE).
- The likelihood function $L_x$ need not to have a maximizer, and even if it does, the maximizer need not be unique. Interest is on getting a good local maximizer which has better properties than the global maximizer.
- Because the log function is monotone, maximizing the likelihood is the same as maximixing the log likelihood.

$$l_x(\theta) = log \quad L_x(\theta)$$

- For many reasons it is more convenient to use log likelihood rather than the likelihood
- The derivatives of the log likelihood function are very important in likelihood theory. The moments of log likelihood derivatives satisfy some important identities.
- If the likelihood is given by

$$L(\theta, X) = h(X)f(X, \theta)$$

then the log likelihood is given by

$$l_x(\theta) = log \quad h(X) + log \quad f(X, \theta)$$

and the derivatives with respect to $\theta$ do not involve $h(X)$ term. - These derivatives are well defined and are the same regardless of $h(X)$ used. The maximizer of this function does not depend on $h(X)$.

- First, the first derivative has expectation zero

$$E_\theta \left\{ \nabla \ l_x(\theta) \right\} = 0$$

where $\nabla \ f$ denotes the vector of partial derivatives of a scalar function $f$ of a vector variable, often called the gradient of $f$, and $\nabla f(x)$ denotes the value of the gradient at the point x.

- Second, the variance of the first derivative is minus the expectation of the second:

$$Var_\theta \left\{ \nabla \ l_x(\theta) \right\} = -E_\theta \left\{ \nabla^2 l_x(\theta) \right\}$$

where $\nabla^2 f$ denotes the matrix of second partial derivatives of a scalar function $f$ of a vector variable, often called the hessian of $f$ and $\nabla^2 f(x)$ denotes the value of the hessian at the point x.

## The Procedure

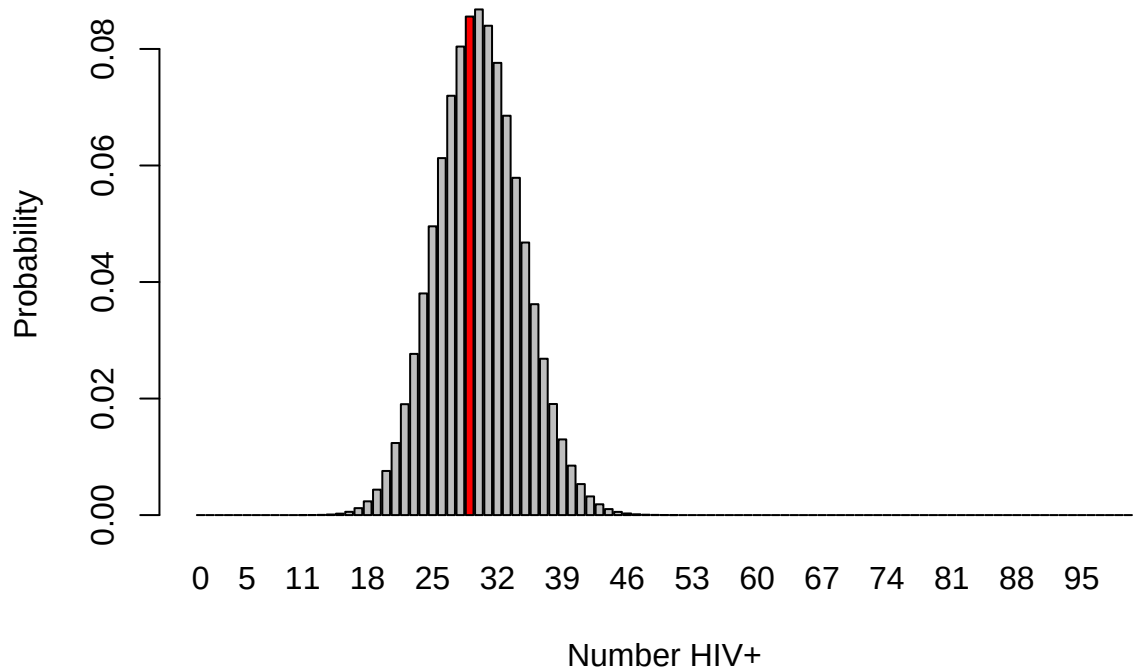
- In most cases that you have encountered so far, we can find the MLE by solving equations in closed form.
- However, life is more complicated than that. . . .by the end of this year this reality will be evident if not already!
- Most real problems do not have closed form solutions due to complicated probability distribution or the non-linearity of the equations obtained from maximum likelihood principles. Computers have become very helpful in this dimension. . . .
- Lets briefly introduce R programming for MLE

## The concept

- In a population of 1,000,000 people with a true prevalence of 30%, the probability distribution of number of positive individuals if 100 are sampled

$$f(x) = \binom{100}{x} (0.3)^x (0.7)^{100-x}$$

```
n <- 100
P <- 0.3
size <- 100
cols <- rep("grey", n + 1)
cols[30] <- "red"
barplot(dbinom(x = 0:n, size = n, prob = 0.3), col = cols, names.arg = 0:size,
    xlab = "Number HIV+", ylab = "Probability")
```

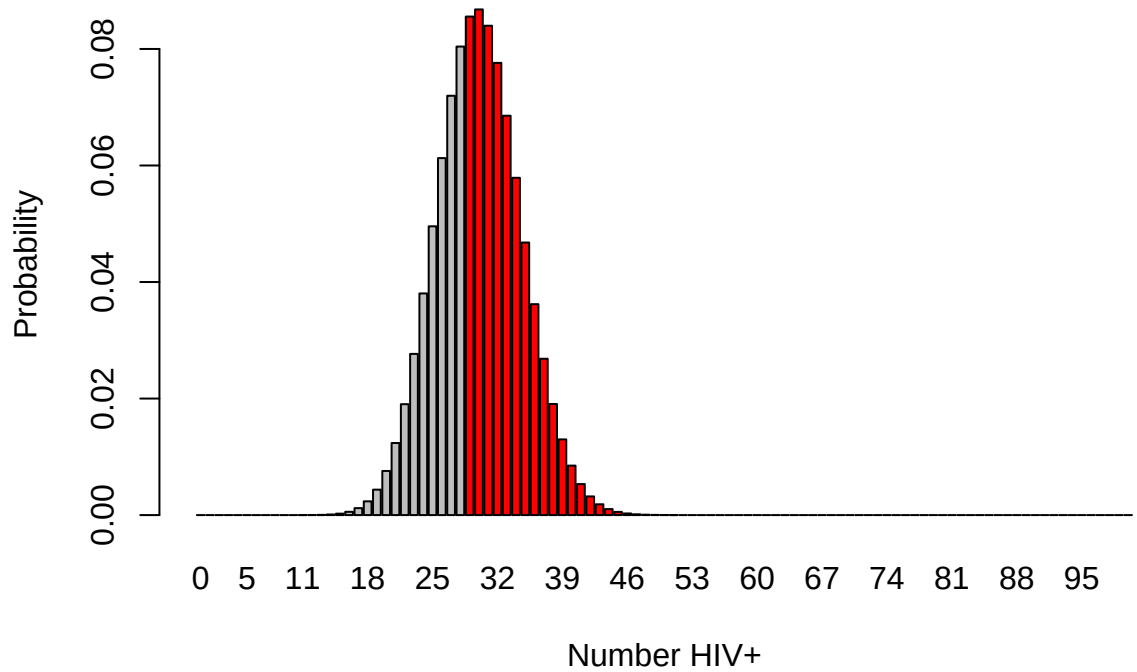



```r
set.seed(1234)
rbinom(n = 1, size = 100, prob = 0.3)
```

```
## [1] 28
```

- We sample 100 people once and 28 are positive
- We do not know the TRUE prevalence, but we can calculate the probability of 28 or a more extreme value occurring for a given prevalence

```r
n <- 100
P <- 0.3
size <- 100
cols <- rep("grey", n + 1)
cols[30:100] <- "red"
barplot(dbinom(x = 0:n, size = n, prob = 0.3), col = cols, names.arg = 0:size,
    xlab = "Number HIV+", ylab = "Probability")
```




```r
pbinom(28, 100, 0.3)  #less than or equal to 28
```

```
## [1] 0.3767782
```

```r
1 - pbinom(27, 100, 0.3)  #Probability of getting 28 or more
```

```
## [1] 0.7036338
```

```r
pbinom(27, 100, 0.3, lower.tail = FALSE)  #Probability of getting 28 or more
```

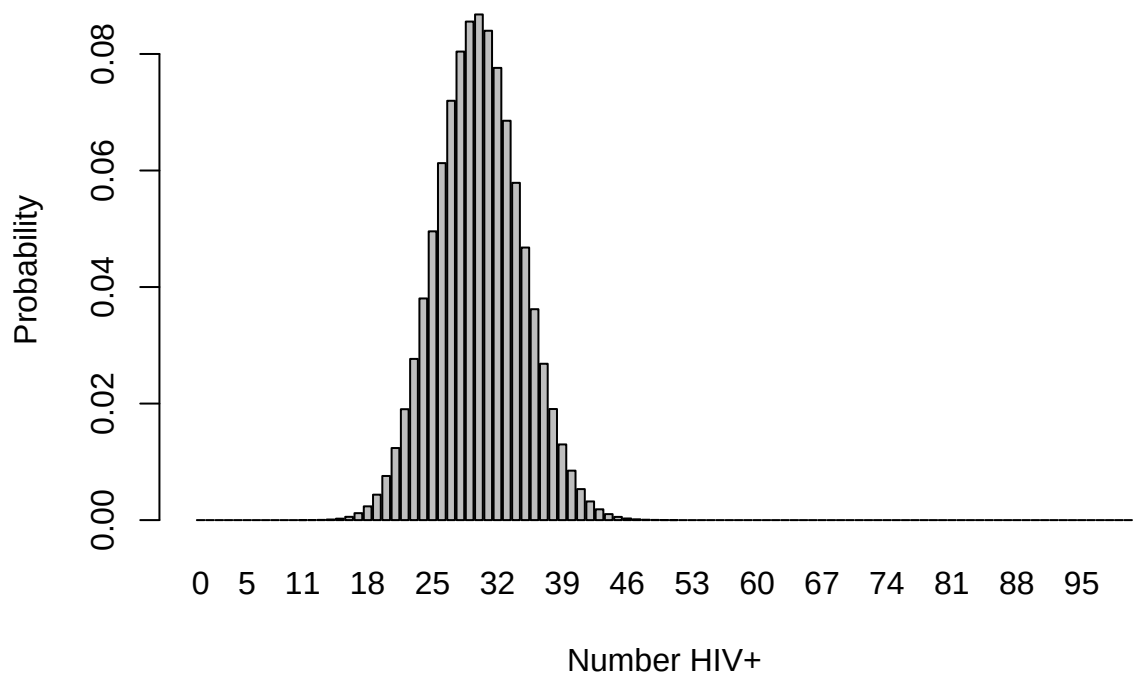
```
## [1] 0.7036338
```

```r
binom.test(28, 100, p = 0.3, alternative = "two.sided")
```

```
##
##  Exact binomial test
##
## data:  28 and 100
## number of successes = 28, number of trials = 100, p-value = 0.7437
## alternative hypothesis: true probability of success is not equal to 0.3
## 95 percent confidence interval:
##  0.1947936 0.3786670
## sample estimates:
## probability of success
##                   0.28
```

- If the true prevalence were 0.15 (15%), then the P(28 or more extreme) is given by:

```
n <- 100
P <- 0.3
size <- 100
cols <- rep("grey", n + 1)
cols[15] <- "red"
barplot(dbinom(x = 0:n, size = n, prob = 0.3), col = cols, names.arg = 0:size,
    xlab = "Number HIV+", ylab = "Probability")
```




```
pbinom(27, 100, 0.15, lower.tail = FALSE)
```

```
## [1] 0.0006129206
```

```
binom.test(28, 100, p = 0.15, alternative = "two.sided")
```

```
##
##  Exact binomial test
##
## data:  28 and 100
## number of successes = 28, number of trials = 100, p-value = 0.001038
## alternative hypothesis: true probability of success is not equal to 0.15
## 95 percent confidence interval:
##  0.1947936 0.3786670
```

```
## sample estimates:
## probability of success
##                   0.28
```

```r
# If true prevalence is 20%
pbinom(27, 100, 0.2, lower.tail = FALSE)
```

```
## [1] 0.03415163
```

```r
binom.test(28, 100, 0.2, alternative = "two.sided")
```

```
##
##  Exact binomial test
##
## data:  28 and 100
## number of successes = 28, number of trials = 100, p-value = 0.05948
## alternative hypothesis: true probability of success is not equal to 0.2
## 95 percent confidence interval:
##  0.1947936 0.3786670
## sample estimates:
## probability of success
##                   0.28
```

```r
# If true prevalence is 25%
pbinom(27, 100, 0.25, lower.tail = FALSE)
```

```
## [1] 0.2776195
```

```r
binom.test(28, 100, 0.25, alternative = "two.sided")
```

```
##
##  Exact binomial test
##
## data:  28 and 100
## number of successes = 28, number of trials = 100, p-value = 0.4891
## alternative hypothesis: true probability of success is not equal to 0.25
## 95 percent confidence interval:
##  0.1947936 0.3786670
## sample estimates:
## probability of success
##                   0.28
```

## An R Example: Normal distribution

- Lets start with a simple example
- Suppose $X_1, \ldots . X_n$ are iid r.v from $N(\mu,\ \sigma^2)$. This distribution has two parameters, ie, $\mu\ and\ \ \sigma^2$.
- The likelihood can be derived using procedures discussed earlier and we get:

$$\sum_{i=1}^{n} \frac{-(X_i - \mu)^2}{2\sigma^2} - \frac{1}{2} log 2\pi - \frac{1}{2} log\ \sigma^2 + log\ dX_i$$

- The 2 terms that are not functions of either $\mu$ *or* $\sigma^2$ can be dropped as they are constants with respect to these parameters.
- In R lets store the X values:

```
Xvec <- c(2, 5, 3, 7, -3, -2, 0)  #We can try any numbers
# here but for our purposes, lets use these!
```

- Then we can define a function in R (The function is the negative of the log likelihood function above).

### The R function for MLE

```
fn <- function(theta) {
    sum(0.5 * (Xvec - theta[1])^2/theta[2] + 0.5 * log(theta[2]))
}
```

- We have two parameters theta[1] and theta[2] which are components of a vector theta.
- We can then use this function to find the values of $\mu$ and $\sigma^2$ that maximize the log likelihood function (minimum for the negative likelihood).
- from the stats4 package we can use the function nlm or optim to get the estimates of theta[1] and theta[2].

### Estimation using nlm

```
nlm(fn, theta <- c(0, 1), hessian = TRUE)
```

```
## $minimum
## [1] 12.00132
##
## $estimate
## [1]  1.714285 11.346934
##
## $gradient
## [1] -4.144835e-08  2.269968e-08
##
## $hessian
##               [,1]         [,2]
## [1,]  6.169068e-01 -4.61169e-06
## [2,] -4.611690e-06  2.71730e-02
##
## $code
## [1] 1
##
## $iterations
## [1] 19
```

```
# or

# optim(theta<-c(0,1), fn, hessian=TRUE) We can start with
```

```r
# any initial values but for our purposes we start with
# theta[1]=0 & theta[2]=1 Hessian matrix tells us something
# about the variance of parameters, or if they are more,
# the variance-covariance matrix of parameters.  The matrix
# contains the second-order partial derivatives of the
# likelihood function evaluated at the maximum likelihood
# estimate
```

### Estimation using optim

```r
optim(theta <- c(0, 1), fn, hessian = TRUE)
```

```
## $par
## [1]  1.714538 11.354415
##
## $value
## [1] 12.00132
##
## $counts
## function gradient
##       55       NA
##
## $convergence
## [1] 0
##
## $message
## NULL
##
## $hessian
##               [,1]          [,2]
## [1,]  6.165003e-01 -1.372058e-05
## [2,] -1.372058e-05  2.711229e-02
```

### An R example: Poisson distribution

- Consider a Poisson distribution for modelling light traffic as used by traffic engineers, based on the rationale that if the rate is approximately constant and the traffic is light (so that individual cars move independently of each other), the distribution of counts of cars in a given time interval or space area should be nearly Poisson (Gerlough and Schuhl 1955).
- The following table shows the number of right turns during 300 3 minute intervals at a specific intersection:
- Supposing the Poisson model is a good model for this dataset, we need to find which Poisson model, that is, estimate the parameter $\lambda$ in the model.

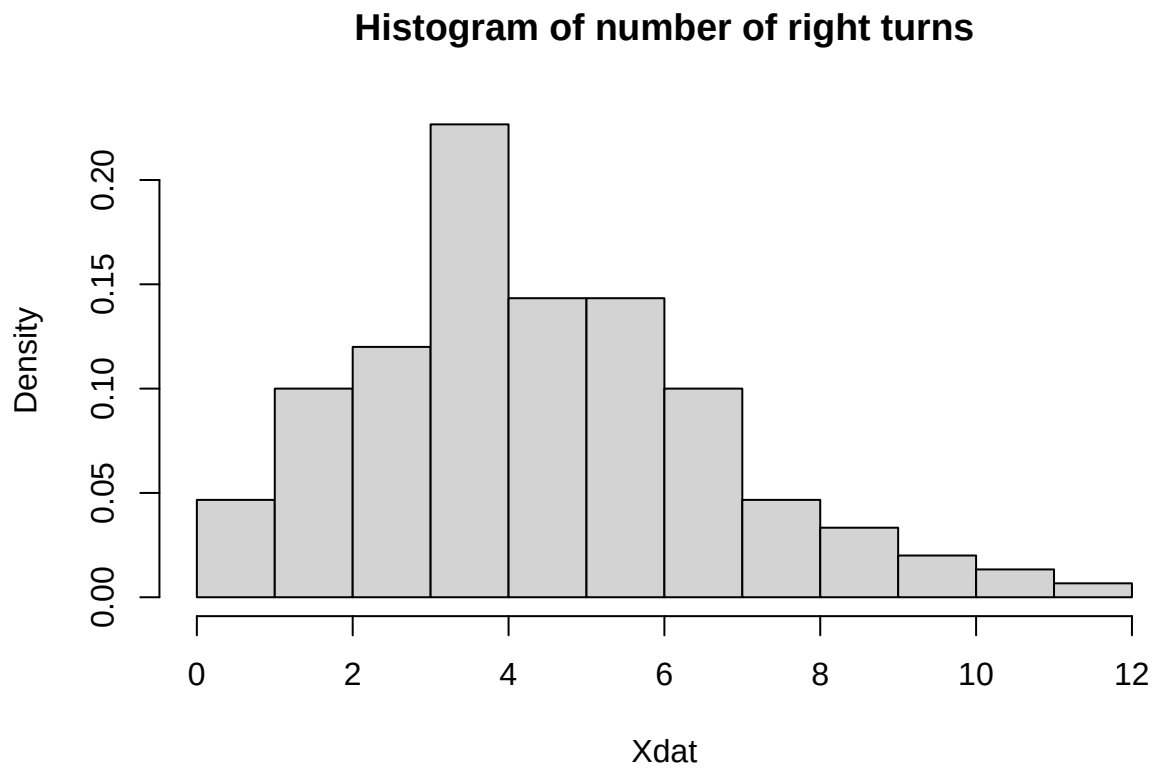
$$P(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}$$

- NB: From the procedure we know we can find this ML estimate of the parameter as: $\hat{\lambda} = \bar{X}$. - We can compare this estimate with the one we obtain from R - Lets enter the data in R as a first step:

Table 1: Number of right turns during 300 3-minute intervals

| n | frequency |
|---|---|
| 0 | 14 |
| 1 | 30 |
| 2 | 36 |
| 3 | 68 |
| 4 | 43 |
| 5 | 43 |
| 6 | 30 |
| 7 | 14 |
| 8 | 10 |
| 9 | 6 |
| 10 | 4 |
| 11 | 1 |
| 12 | 1 |
| 13 | 0 |

```r
Xdat <- c(rep(0, 14), rep(1, 30), rep(2, 36), rep(3, 68), rep(4,
    43), rep(5, 43), rep(6, 30), rep(7, 14), rep(8, 10), rep(9,
    6), rep(10, 4), rep(11, 1), rep(12, 1))

## Plotting histogram for the data

hist(Xdat, main = "Histogram of number of right turns", right = FALSE,
    prob = TRUE)
```




```
# size of the sample:
n <- length(Xdat)
n
```

```
## [1] 300
```

- To obtain the MLE, we need to maximize the likelihood function or the log likelihood as highlighted before. The R package provides a function which can minimize an object function, thus we can define the negative log likelihood function as:

```
negLL <- function(lambda) {
    n * lambda - sum(Xdat) * log(lambda) + sum(log(factorial(Xdat)))
}

# nLL<- function(lambda): defines the function and function
# name is nLL lambda: The input parameter of the function
# function: A key word in R which means following is a
# function lambda: this argument can also be vector as in
# previous example The statement inside the curly brackets
# is the body of the function. The only unknown should be
# lambda!
```

- Afetr defining the negative log likelihood, we can perform the optimization as:

```r
mleout <- nlm(negLL, p = c(0.5), hessian = TRUE)
```

```
## Warning in log(lambda): NaNs produced

## Warning in nlm(negLL, p = c(0.5), hessian = TRUE): NA/NaN replaced by maximum
## positive value

## Warning in log(lambda): NaNs produced

## Warning in nlm(negLL, p = c(0.5), hessian = TRUE): NA/NaN replaced by maximum
## positive value
```

```r
mleout
```

```
## $minimum
## [1] 667.183
##
## $estimate
## [1] 3.893331
##
## $gradient
## [1] -2.569636e-05
##
## $hessian
##          [,1]
## [1,] 77.03948
##
## $code
## [1] 1
##
## $iterations
## [1] 10
```

```r
# nlm: Is the nonlinear minimization function provided in R
# First argument is the object function to be minimized
# p=c(0.5) specifies the initial value of the unknown
# parameter hessian=TRUE: says we want a HESSIAN matrix as
# a part of the output
```

## Cauchy distribution: Small background on Cauchy distributions

- The standard Cauchy distribution is the continuous univariate distribution having density

$$f(x) = \frac{1}{\pi} \cdot \frac{1}{1+x^2} \qquad -\infty < X < \infty$$

  NB: This distribution has no parameters, but induces a two parameter location scale family having densities

$$f_{\mu,\sigma^2}(x) = \frac{1}{\sigma}.f\left(\frac{x-\mu}{\sigma}\right)$$

- If $f$ is any distribution having mean zero and variance 1, then $f_{(\mu,\sigma)}$ has mean $\mu$ and variance $\sigma^2$. - But the Cauchy distribution has neither mean nor variance. - Thus we call $\mu$ the location parameter and $\sigma$ the scale parameter. - $\mu$ is then the population median and a robust scale estimator for the median is the IQR - For a standard Cauchy, the IQR is given by:

```
qcauchy(3/4) - qcauchy(1/4)
```

```
## [1] 2
```

- The population IQR of $Cauchy(\mu, \sigma^2)$ distribution is $2\sigma$ hence a good estimator of $\sigma$ is the IQR divided by 2.

### Cauchy Location Model (one parameter)

- For this model $\mu$ is unknown but $\sigma = 1$ is known.
- The negative log likelihood can be written as

```
mlogl <- function(mu, X) {

    sum(-dcauchy(X, location = mu, log = TRUE))
}
# dcauchy calls the cauchy distribution and log=TRUE takes
# the log of the likelihood

# simulate data
n <- 30
set.seed(42)
X <- rcauchy(n)
# Starting values
mu.start <- median(X)
mu.start
```

```
## [1] -0.1955062
```

```
# MLE estimate function

outmle2 <- nlm(mlogl, mu.start, X = X)
mu.hat <- outmle2$estimate
mu.hat
```

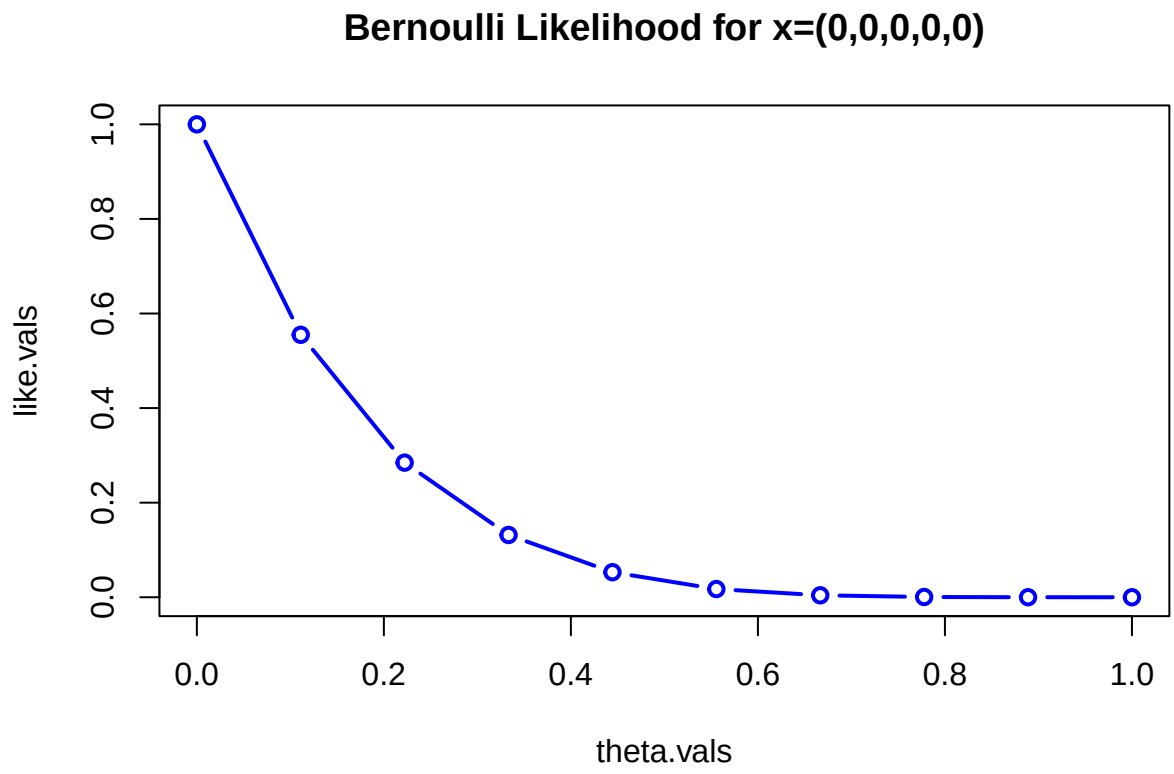
```
## [1] -0.1816501
```

## Plotting Likelihoods

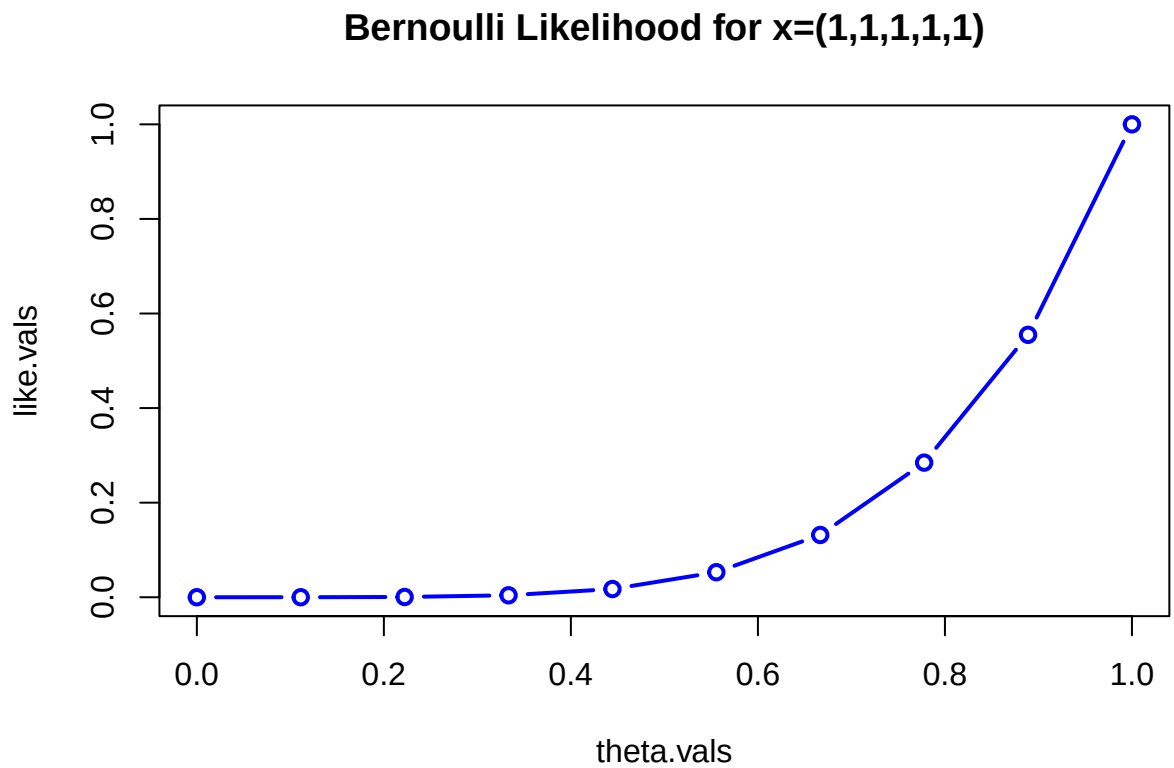
```r
options(digits = 4, width = 70)
library(maxLik)
```

```
## Warning: package 'maxLik' was built under R version 4.5.2

## Loading required package: miscTools

## Warning: package 'miscTools' was built under R version 4.5.2

##
## Please cite the 'maxLik' package as:
## Henningsen, Arne and Toomet, Ott (2011). maxLik: A package for maximum likelihood estimation in R. Co
##
## If you have questions, suggestions, or comments regarding the 'maxLik' package, please use a forum or
## https://r-forge.r-project.org/projects/maxlik/
```

```r
# Bernoulli example

# Bernoulli likelihood x = 0, 1 f(x,theta) =
# (theta^x)*(1-theta)^(1-x)

likelihood.Bernoulli = function(theta, x) {
    # theta success probability parameter x vector of data
    n = length(x)
    ans = theta^sum(x) * (1 - theta)^(n - sum(x))
    return(ans)
}

# plot Bernoulli likelihood
x = rep(0, 5)
theta.vals = seq(0, 1, length.out = 10)
like.vals = likelihood.Bernoulli(theta.vals, x)
plot(theta.vals, like.vals, type = "b", col = "blue", lwd = 2,
    main = "Bernoulli Likelihood for x=(0,0,0,0,0)")
```

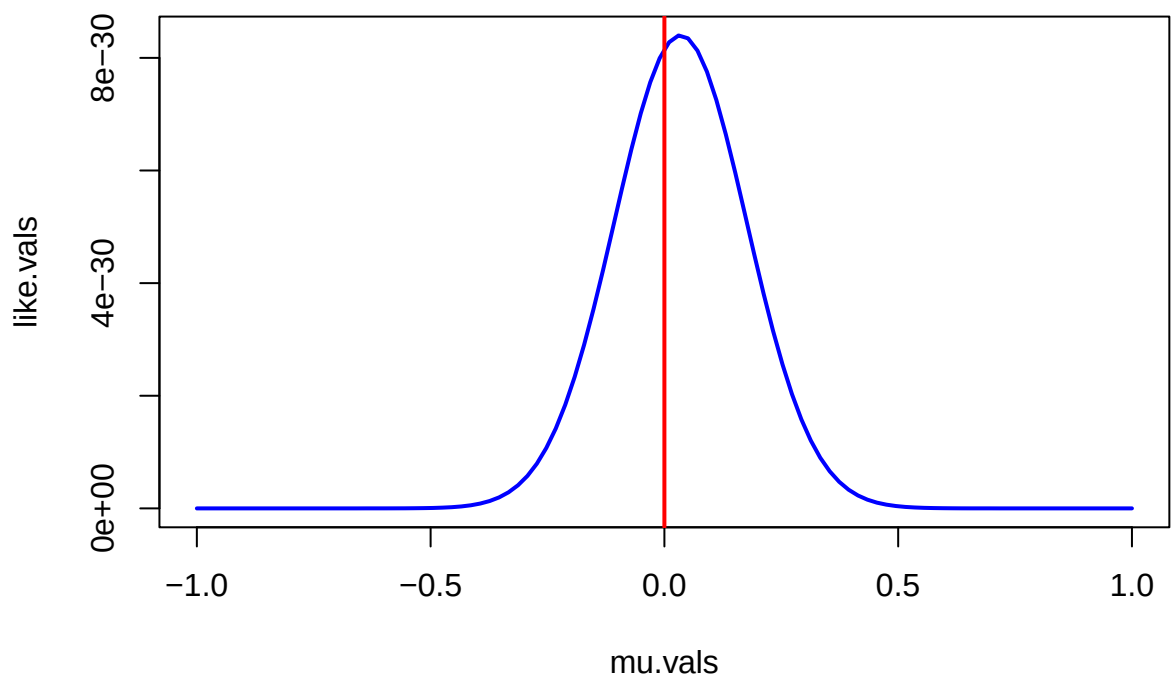



```
x = rep(1, 5)
like.vals = likelihood.Bernoulli(theta.vals, x)
plot(theta.vals, like.vals, type = "b", col = "blue", lwd = 2,
    main = "Bernoulli Likelihood for x=(1,1,1,1,1)")
```




```
# normal likelihood

likelihood.normal = function(theta, x) {
    # theta vector of normal distribution parameters theta
    # = (mu, sig2)' x vector of data
    mu = theta[1]
    sig2 = theta[2]
    n = length(x)
    a1 = (2 * pi * sig2)^-(n/2)
    a2 = -1/(2 * sig2)
    y = (x - mu)^2
    ans = a1 * exp(a2 * sum(y))
    return(ans)
}

likelihood.normal.mu = function(mu, sig2 = 1, x) {
    # mu mean of normal distribution for given sig x vector
    # of data
    n = length(x)
    a1 = (2 * pi * sig2)^-(n/2)
    a2 = -1/(2 * sig2)
    y = (x - mu)^2
    ans = a1 * exp(a2 * sum(y))
    return(ans)
}
```

```r
# generate N(0,1) data
n = 50
set.seed(123)
x = rnorm(n, mean = 0, sd = 1)

# compute normal likelihood as function of mu
mu.vals = seq(-1, 1, length.out = 100)
like.vals = rep(0, length(mu.vals))
for (i in 1:length(like.vals)) {
    like.vals[i] = likelihood.normal.mu(mu.vals[i], sig2 = 1,
        x = x)
}

plot(mu.vals, like.vals, type = "l", col = "blue", lwd = 2)
abline(v = 0, col = "red", lwd = 2)
```

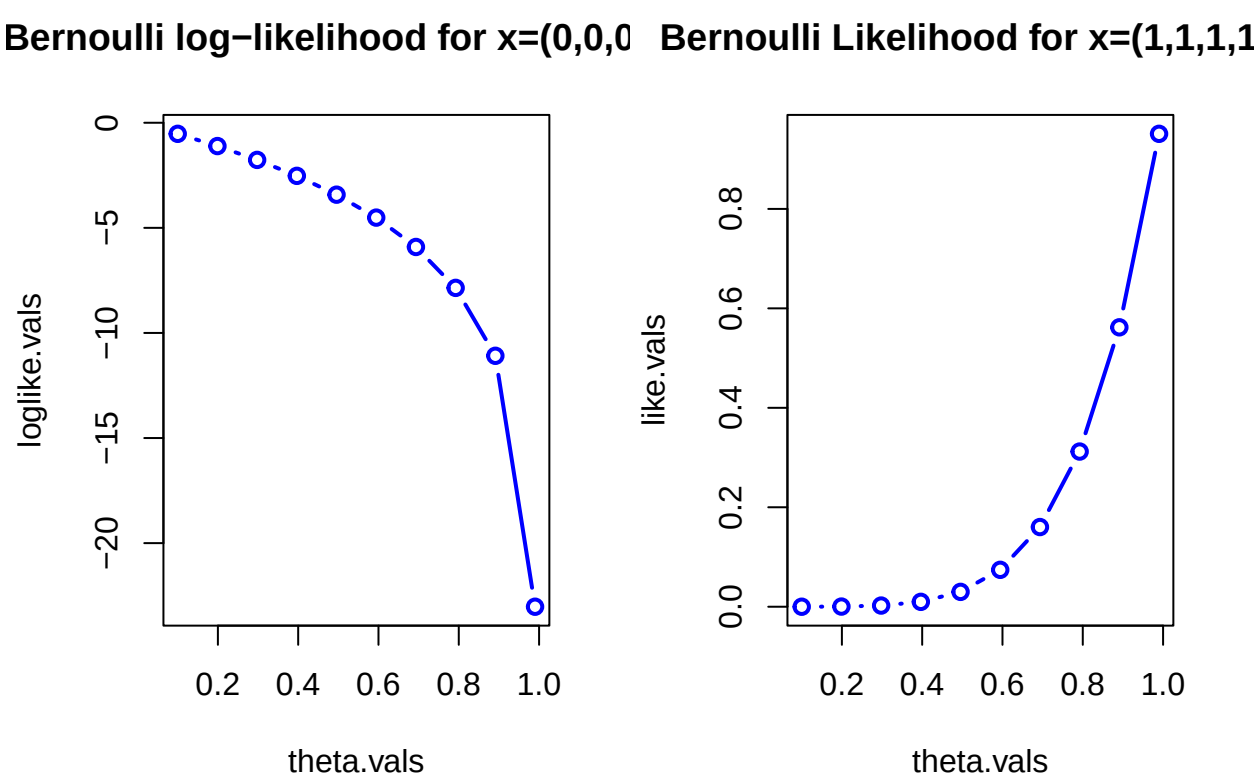



```r
mean(x)
```

```
## [1] 0.0344
```

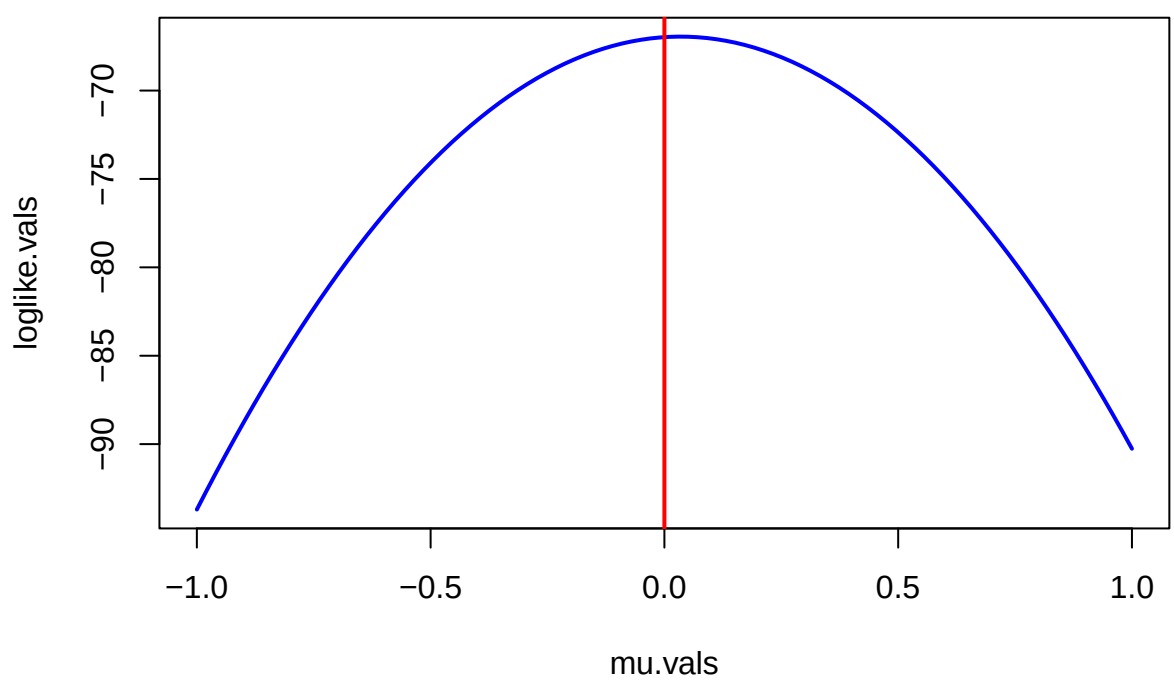
```r
# log-likelihood functions

# Bernoulli log-likelihood x = 0, 1 f(x,theta) =
# (theta^x)*(1-theta)^(1-x)
```

```r
loglike.Bernoulli = function(theta, x) {
    # theta success probability parameter x vector of data
    n = length(x)
    ans = log(theta) * sum(x) + log(1 - theta) * (n - sum(x))
    return(ans)
}

# plot Bernoulli log-likelihood
par(mfrow = c(1, 2))
x1 = rep(0, 5)
theta.vals = seq(0.1, 0.99, length.out = 10)
loglike.vals = loglike.Bernoulli(theta.vals, x1)
plot(theta.vals, loglike.vals, type = "b", col = "blue", lwd = 2,
    main = "Bernoulli log-likelihood for x=(0,0,0,0,0)")

x2 = rep(1, 5)
like.vals = likelihood.Bernoulli(theta.vals, x2)
plot(theta.vals, like.vals, type = "b", col = "blue", lwd = 2,
    main = "Bernoulli Likelihood for x=(1,1,1,1,1)")
```




```r
par(mfrow = c(1, 1))

# normal log-likelihood
loglike.normal.mu = function(mu, sig2 = 1, x) {
    # mu mean of normal distribution for given sig x vector
```

```r
    # of data
    n = length(x)
    a1 = -(n/2) * log(2 * pi) - (n/2) * log(sig2)
    a2 = -1/(2 * sig2)
    y = (x - mu)^2
    ans = a1 + a2 * sum(y)
    return(ans)
}

# generate N(0,1) data
n = 50
set.seed(123)
x = rnorm(n, mean = 0, sd = 1)

# compute normal likelihood as function of mu
mu.vals = seq(-1, 1, length.out = 100)
loglike.vals = rep(0, length(mu.vals))
for (i in 1:length(loglike.vals)) {
    loglike.vals[i] = loglike.normal.mu(mu.vals[i], sig2 = 1,
        x = x)
}

plot(mu.vals, loglike.vals, type = "l", col = "blue", lwd = 2)
abline(v = 0, col = "red", lwd = 2)
```



```r
# Optimization in R

# optimize() function
test.fun = function(x) {
    return(x^2)
}

ans = optimize(test.fun, lower = -1, upper = 1, maximum = FALSE)
class(ans)
```

```
## [1] "list"
```

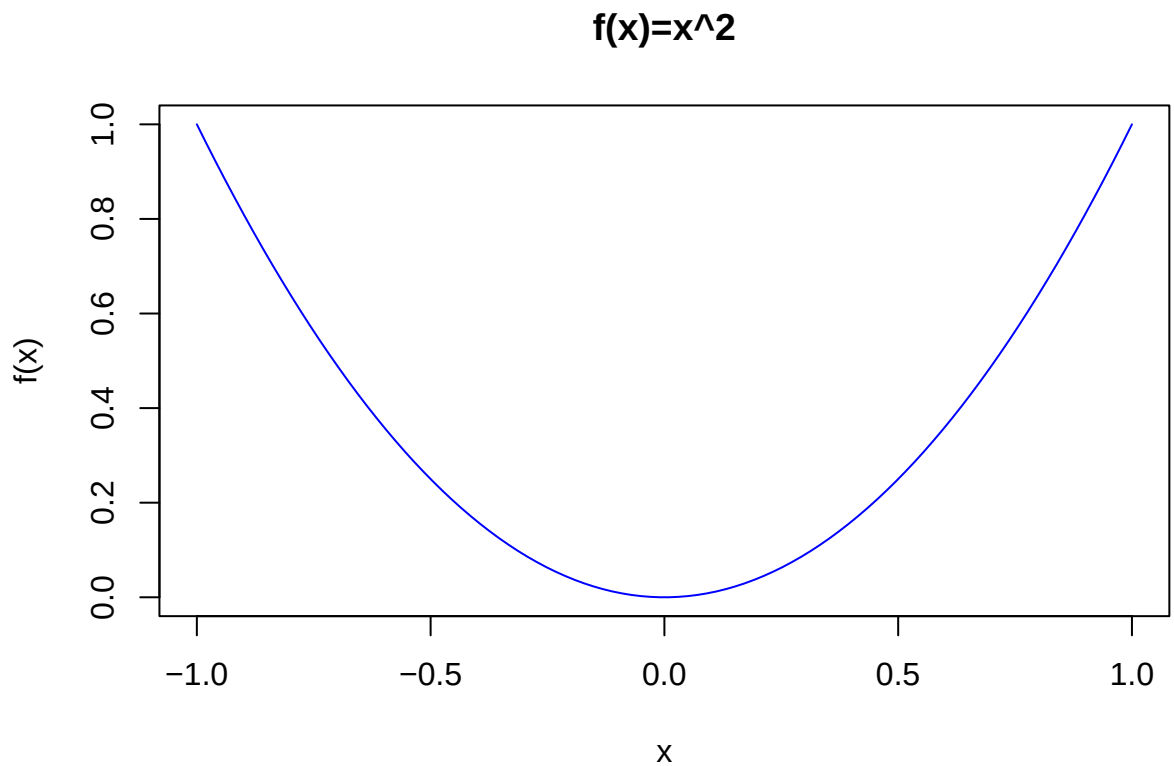
```r
names(ans)
```

```
## [1] "minimum"   "objective"
```

```r
ans
```

```
## $minimum
## [1] -2.776e-17
##
## $objective
## [1] 7.704e-34
```

```r
x.vals = seq(-1, 1, length.out = 100)
plot(x.vals, test.fun(x.vals), type = "l", col = "blue", xlab = "x",
    ylab = "f(x)", main = "f(x)=x^2")
```




```r
# use optim() to minimize functions of multiple variables
test.fun = function(theta) {
    ans = theta[1]^2 + theta[2]^2
    return(ans)
}

# set starting values for optimizer
theta.start = c(1, 1)
ans = optim(par = theta.start, fn = test.fun, method = "BFGS")
class(ans)
```

```
## [1] "list"
```

```r
names(ans)
```

```
## [1] "par"         "value"       "counts"      "convergence"
## [5] "message"
```

```r
ans
```

```
## $par
## [1] -4.264e-16 -4.264e-16
##
## $value
```

```
## [1] 9.088e-30
##
## $counts
## function gradient
##        8        3
##
## $convergence
## [1] 0
##
## $message
## NULL
```

```r
# maximize normal log-likelihood using optim by minimizing
# -1*log-likelihood
loglike.normal = function(theta, x) {
    # theta parameters c(mu,sig2) x vector of data
    mu = theta[1]
    sig2 = theta[2]
    n = length(x)
    a1 = -(n/2) * log(2 * pi) - (n/2) * log(sig2)
    a2 = -1/(2 * sig2)
    y = (x - mu)^2
    ans = a1 + a2 * sum(y)
    # return -1 * loglike
    return(-ans)
}

# generate N(0,1) data
n = 50
set.seed(123)
x = rnorm(n, mean = 0, sd = 1)
# set starting values for optimizer
theta.start = c(0, 1)
ans = optim(par = theta.start, fn = loglike.normal, x = x, method = "BFGS")
```

```
## Warning in log(sig2): NaNs produced

## Warning in log(sig2): NaNs produced
```

```r
ans$par
```

```
## [1] 0.03442 0.84011
```

```r
# verify mle is mean and scaled variance
mean(x)
```

```
## [1] 0.0344
```

```r
var(x) * (n - 1)/n
```

```
## [1] 0.8401
```

```r
# compute MLE and get standard errors set starting values
# for optimizer
ans = optim(par = theta.start, fn = loglike.normal, x = x, method = "BFGS",
    hessian = TRUE)
```

```
## Warning in log(sig2): NaNs produced
## Warning in log(sig2): NaNs produced
```

```r
names(ans)
```

```
## [1] "par"         "value"       "counts"      "convergence"
## [5] "message"     "hessian"
```

```r
ans$hessian
```

```
##            [,1]       [,2]
## [1,] 59.5160231 -0.0009716
## [2,] -0.0009716 35.4202372
```

```r
se.mle = sqrt(diag(solve(ans$hessian)))
se.mle
```

```
## [1] 0.1296 0.1680
```

```r
# use maxLik function from maxLik package here function to
# compute log-likelihood returns log-likelihood values and
# not -1*log-likelihood values
library(maxLik)
loglike.normal = function(theta, x) {
    # theta parameters c(mu,sig2) x vector of data
    mu = theta[1]
    sig2 = theta[2]
    n = length(x)
    a1 = -(n/2) * log(2 * pi) - (n/2) * log(sig2)
    a2 = -1/(2 * sig2)
    y = (x - mu)^2
    ans = a1 + a2 * sum(y)
    return(ans)
}
theta.start = c(0, 1)
names(theta.start) = c("mu", "sig2")
theta.mle = maxLik(loglike.normal, start = theta.start, x = x)
class(theta.mle)
```

```
## [1] "maxLik" "maxim"  "list"
```

```r
names(theta.mle)
```

```
##  [1] "maximum"     "estimate"    "gradient"    "hessian"
##  [5] "code"        "message"     "last.step"   "fixed"
##  [9] "iterations"  "type"        "control"     "objectiveFn"
```

```r
theta.mle
```

```
## Maximum Likelihood estimation
## Newton-Raphson maximisation, 5 iterations
## Return code 1: gradient close to zero (gradtol)
## Log-Likelihood: -66.59 (2 free parameter(s))
## Estimate(s): 0.0344 0.8401
```

```r
summary(theta.mle)
```

```
## --------------------------------------------
## Maximum Likelihood estimation
## Newton-Raphson maximisation, 5 iterations
## Return code 1: gradient close to zero (gradtol)
## Log-Likelihood: -66.59
## 2  free parameters
## Estimates:
##      Estimate Std. error t value Pr(> t)
## mu     0.0344     0.1296    0.27    0.79
## sig2   0.8401     0.1680    5.00 5.8e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## --------------------------------------------
```

# Minimum variance Unbiased Estimators

## Minimum Variance criterion

- In searching for optimal estimators we need to adopt some optimality criterion.
- The natural criterion to use in thisis the Mean Square Error (MSE) defined as:

$$MSE(\hat{\boldsymbol{\theta}}) = E\left[\left(\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}\right)^2\right]$$

- The downside to adopting this natuaral approach is it leads to unrealizable estimators, that is, the ones that cannot be written sorely as a function of the data. - The MSE decomposition:

$$\begin{aligned} MSE(\hat{\boldsymbol{\theta}}) &= E\left\{\left[\left(\hat{\theta} - E(\hat{\theta})\right) + \left(E(\hat{\theta}) - \theta\right)\right]^2\right\} \\ &= E\left(\hat{\theta} - E(\hat{\theta})\right)^2 + E\left(E(\hat{\theta} - \theta\right)^2 \\ &= Var(\hat{\boldsymbol{\theta}}) + b^2(\hat{\boldsymbol{\theta}}) \end{aligned}$$

which shows that the MSE is composed of errors due to the variance of the estimator as well las the bias. - Consider an estimator for the mean given by say:

$$\hat{\mu} = \frac{a}{n}\sum_{i=1}^{n} X_i$$

where $a$ is is some unknown constant. - The $E(\hat{\mu}) = a\mu$ and $Var(\hat{\mu}) = \frac{a^2\sigma^2}{n}$ and the $bias = E(\hat{\mu}) - \mu = (a-1)\mu$ hence $(bias)^2 = (a-1)^2\mu^2$.

- Therefore the
$$MSE(\hat{\mu}) = \frac{a^2\sigma^2}{n} + (a-1)^2\mu^2$$
and to find the optimal value of $a$ that minimizes the MSE, we differentiate with respect to $a$ and equate to zero.

$$\frac{d(MSE(\mu))}{da} = \frac{a\sigma^2}{n} + a\mu^2 - \mu^2 = 0$$

such that $a = \frac{\mu^2}{\frac{\sigma^2}{n}+\mu^2}$, thus the optimal value of $a$ depends on the unkown parameter $\mu$. The implication is $a$ is indeterminable! - The estimator is therefore not realizable! -Any criteria which depends on the bias leads to an unrealizable estimator and hence practically this method does not always work thus the need for alternatives. -The alternative approach is to constrain the bias to zero, and find the estimator which minimizes the variance. Such an estimator is termed known as the MINIMUM VARIANCE UNBIASED estimator.

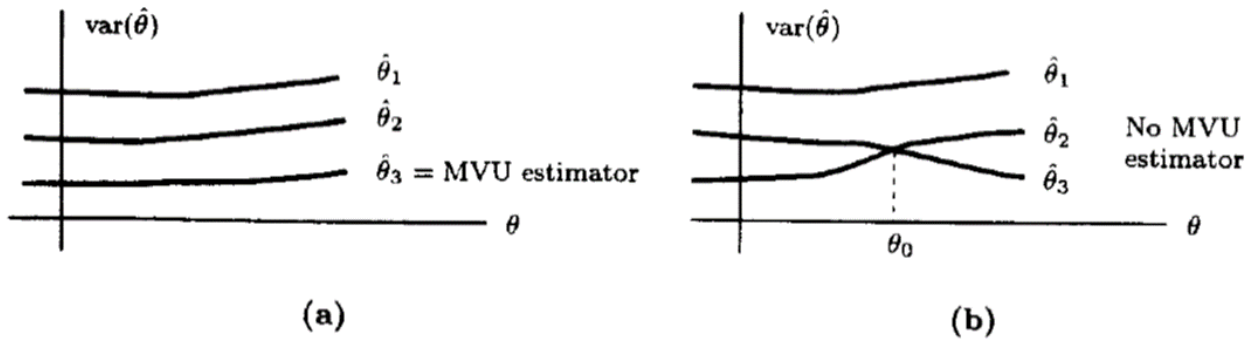



**Figure 2.3** Possible dependence of estimator variance with $\theta$

- Note that the MSE for an unbiased estimator is the variance.

## Minimum VAriance Estimators

- The question is: Is there a "best" estimator, in the sense of possessing a minimum variance? How do we know if an estimator is "best"?

## The Cramer-Rao Inequality and Lower bound

- The Cramer-Rao Lower Bound (CRLB) sets a lower bound on the variance of any unbiased estimator.
- Useful as follows
  1. If we find an estimator that achieves the CRLB, then we know that we have found a Minimum Variance Unbiased Estimator (MVUE).
  2. The CRLB can provide a benchmark against which we can compare the performance of any unbiased estimator.
  3. The CRLB can be used to rule out impossible estimators.
  4. The theory behind the CRLB can tell us if an estimator exists that achieves the lower bound

**Theorem: The Cramer-Rao Inequality**

- Let $X_1, X_2, \ldots, X_n$ be a random sample from $f_X(x, \theta)$, where $f(x, \theta)$ has continuous first order and second order partial derivatives at all but a finite set of points. Suppose the set of $x's$ for which $f_X(x, \theta) \neq 0$ does not depend on $\theta$.
- Let $\hat{\theta} = h(X_1, \ldots, X_n)$ be an unbiased estimatimator of $\theta$. Then

$$Var(\hat{\theta}) \geq \frac{1}{nE\left[\left(\frac{\partial\ ln\ f_X(x,\theta)}{\partial\theta}\right)^2\right]}$$

and

$$Var(\hat{\theta}) \geq \frac{1}{nE\left[\frac{\partial^2 ln f_X(x,\theta)}{\partial^2\theta}\right]}$$

## Example

- Let $X_1, \ldots, X_n$ be the total number of successes in each of $n$ independent trials, where $p = Prob\ of\ success$ at any given trial is an unknown parameter. We have that:

$$P_X(k; p) = p^k(1-p)^{1-k} \quad k = 0, 1; \quad 0 < p < 1$$

and $X = X_1 + X_2 + \cdots + X_n = Total\ number\ of\ successes.$ Define

$$\hat{p} = \frac{\sum_{i=1}^{n} X_i}{n} = \frac{X}{n}$$

1. Show that $\hat{p}$ is unbiased
2. Compare $Var(\hat{p})$ with the CRLB for $P_X$.

# Statistics and Sufficient Statistics

- Suppose we have a random sample $X_1, X_2, \ldots, X_n$ taken from a distribution $f(x|\theta)$ which relies on an unknown parameter $\theta$ in a parameter space $\Theta$.
- The purpose of parameter estimation is to estimate the parameter $\theta$ from the random sample.
- We have so far studied two parameter estimation methods, that is, method of moments and maximum likelihood. Tomorrow you will be introduced to the Bayesian estimation method.
- It can be noted from these methods that the estimators can be expressed as a function of the random sample $X_1, \ldots, X_n$. This function is normally referred to as a statistic.
- Formally, any real-valued function $T = h(X_1, \ldots, X_n)$ of the observations in the sample is called a statistic.
- In this function, there should not be any unknown parameter.
- For example, suppose we have a random $X_1, \ldots, X_n$, then $\bar{X},\ max(X_1, \ldots, X_n),\ Median(X_1, \ldots, X_n)\ and\ h(X_1, \ldots, X_n$ are statistics; however$X_1 + \mu$ is not a statistic if $\mu$ is unknown.
- For parameter estimation problem, we know nothing about the parameter but the observations from such a distribution.

- The observations $X_1, \ldots, X_n$ is our first hand of information source about the parameter, ie, all the available information about the parameter is contained in the observations.
- However, we know that the estimators we obtained are always functions of the observations, ie, the estimators are statistics eg sample mean, sample variances etc.
- In some sense, this process can be thought as a "compression" of the original observation data: initially we have $n$ numbers, but after this "compression", we only have 1 number.
- This "compression" always makes us lose information about the parameter, can never make us obtain more information.
- The best case is that this "compression" result containss the same amount of information as the information contained in the $n$ observations.
- Sunch a statistic is called a sufficient statistic.
- Thus a sufficient statistic "absorbs" all the available information about $\theta$ contained in the sample.
- If $T(X_1, \ldots, X_n)$ is a statistic and $t$ is a particular value of $T$, then the conditional joint distribution of $X_1, \ldots, X_n$ given that $T = t$ can be calculated.
- In general, this joint conditional distribution will depend on the value of $\theta$. Therefore, for each value of $t$, there will be a family of possible conditional distributions corresponding to the different possible values of $\theta \in \Theta$.
- However, it may happen that for each possible value of $t$, the conditional joint distribution of $X_1, \ldots, X_n$ given that $T = t$ is the same for all the values of $\theta \in \Theta$ and therefore does not actually depend on the value of $\theta$.
- In this case, we say that $T$ is a sufficient statistic for the parameter $\theta$.
- Formally, a statistic $T(X_1, \ldots, X_n)$ is said to be a sufficient statistic for $\theta$ if the conditional distribution of $X_1, \ldots, X_n$ given $T = t$, does not depend on $\theta$ for any value of $t$.
- In other words, given the value of $T$, we can gain no more knowledge about $\theta$ from knowing more about the probability distribution of $X_1, \ldots, X_n$.
- We could envision keeping only $T$ and throwing away all the $X_i$ without losing any information!
- The concept of sufficiency arises as an attempt to answer the question: Is there a statistic ie a function $T(X_1, \ldots, X_n)$, that contains all the information in the sample about $\theta$? If so, a reduction or compression of the original data to this statistic without loss of information is possible.
- The following example illustrates the concept:
- Consider a coin flipped 4 times. The probability, $p$, of success (getting a head) is unknown. Suppose that the observed outcomes of the tosses are $\{1, 1, 0, 0\}$ where $1 = Head\ and\ 0 = Tail$.
- Based on MLE, the estimate for $\hat{p} = \frac{numbersuccess}{samplespace} = \frac{2}{4} = \frac{1}{2} = 0.5$
- Let's compute some conditional probabilities:

$$P((1,1,0,0)|\hat{p} = 0.5) = \frac{P(1,1,0,0)\ and\ \hat{p} = 0.5}{P(\hat{p} = 0.5)}$$

$$= \frac{P(1,1,0,0)}{2\ successes\ in\ 4\ trials} = \frac{p^2(1-p)^2}{\binom{4}{2}p^2(1-p)^2} = \binom{4}{2}^{-1}$$

- Hence $P((1,1,0,0)|\hat{p} = 0.5) = \binom{4}{2}^{-1}$

- Similarly, we may verify that

$$P((1,0,1,0)|\hat{p}=0.5) = \binom{4}{2}^{-1}$$
$$P((1,0,0,1)|\hat{p}=0.5) = \binom{4}{2}^{-1}$$

$$P((0,1,1,0)|\hat{p}=0.5) = \binom{4}{2}^{-1}$$
$$P((0,1,0,1)|\hat{p}=0.5) = \binom{4}{2}^{-1}$$
$$P((0,0,1,1)|\hat{p}=0.5) = \binom{4}{2}^{-1}$$

- So knowing what a particular outcome is does not add any additional information to what we know about $p$, once we have been informed that $\hat{p} = 0.5$.
- In general, Let $X_1, \ldots, X_n$ be a sequence of independent bernoulli trials with $P(X_i = 1) = \theta$. We can verify that $T = \sum_i^n X_i$ is a sufficient statistic for $\theta$.
- Proof:

$$P(X_1 = x_1, \ldots, X_n = x_n | T = t) = \frac{P((X_1 = x_1, \ldots, X_n = x_n)}{P(T = t)}$$

- NB: The $X_i$ can take only $0s$ *or* $1s$, the probability in the numerator is the probability that some particular set of $t$, $X_i$ are equal to 1s, and the other $n - t$ are 0s.
- Since the $X_i s$ are independent, the probability of this is

$$\theta^t (1-\theta)^{n-t}$$

- To find the denominator, note that the distribution of $T$, the total number ones, is binomial with $n$ trials and probability of success $\theta$. Therefore the ratio in the equation above becomes:

$$\frac{\theta^t (1-\theta)^{n-t}}{\binom{n}{t}\theta^t (1-\theta)^{n-t}} = \binom{n}{t}^{-1}$$

- The conditional distribution thus does not involve on $\theta$ at all.

- Given the total number of 1s, the probability that they occur on any particular set of $t$ trials is the same for any value of $\theta$ so that set of trials contains no additional information about $\theta$.
- Theorem: Let $X_1, \ldots, X_n$ be a random sample from $f_X(x|\theta)$. The estimator $\hat{\theta} = h(X_1, \ldots, X_n)$ is said to be sufficient for $\theta$ if for all $\theta$ and all possible sample points, the conditional pdf of $X_1, \ldots, X_n$ given $\hat{\theta}$ does not depend on $\theta$.

## Factorization Theorem

- The above definition of sufficiency is for intuitive purpose and hard to implement because it does not indicate how to go about finding the sufficient statistic.
- A method for finding a sufficient statistic which can be applied in many problems is presented below and known as the Factorization theorem.
- Theorem: Let $X_1, \ldots, X_n$ form a random sample from either a continuous distribution or a discrete distribution for which the pdf or the point mass function is $f(x|\theta)$, where the value of $\theta$ is unknown and belongs to a given parameter space $\Theta$. A statistic $T(X_1, \ldots, X_n)$ is a sufficient statistic for $\theta$ if and only if the joint pdf or the joint point mass function $f_n(X|\theta)$ of $X_1, \ldots, X_n$ can be factorized as follows for all values of $X = (x_1, \ldots, x_n) \in R^n$ and all values of $\theta \in \Theta$:

$$f_n(X|\theta) = u(X)v\left[T(X), \theta\right]$$

Here, the function u and v are nonnegative, the function u may depend on $X$ but does not depend on $\theta$, and the function v depends on $\theta$ but will depend on the observed values of $X$ only through the value of the statistic $T(X)$.

- Proof for discrete case: Suppose the frequency function can be factored in the given form (Theorem). Let $X = (X_1, \ldots, X_n)$ and $x = (x_1, \ldots, x_n)$, then

$$P(T = t) = \sum_{X:T(X)=t} P(X = x) = \sum_{X:T(X)=t} u(X)v[T(X), \theta] = v(t, \theta) \sum_{X:T(X)=t} u(X)$$

-Thus we can have:

$$P(X = x|T = t) = \frac{P(X = x, T = t)}{P(T = t)} = \frac{u(X)}{\sum_{X:T(X)=t} u(X)}$$

which does not depend on $\theta$, and therefore $T$ is a sufficient statistic.

- Conversely, suppose the conditional distribution of $X$ given $T$ is independent of $\theta$, that is, $T$ is a sufficient statistic. Then we let $u(X) = P(X = x|T = t,\ \theta)$, and let $v(t, \theta) = P(T = t|\theta))$, it then follows that

$$P(X = x|\theta) = P(X = x, T = t|\theta)$$

This is true because $T(X) = t$ is redundant

$$= P(X = x|T(X) = t|\theta)P(T(X) = t|\theta) = u(X)v(T(x_1, \ldots, x_n), \theta)$$

Which gives back the desired form.

- Example: Suppose $X_1, \ldots, X_n$ form a random sample from a Poisson distribution for which the value of the mean $\theta$ is unknown ($\theta > 0$). Show that $T = \sum_{i=1}^n X_i$ is a sufficient statistic for $\theta$.
- Solution: For all $X_1, \ldots, X_n$, the joint probability mass function, $f_n(X|\theta)$ is given by:

$$f_n(X|\theta) = \prod_{i=1}^n \frac{e^{-\theta}\theta^{x_i}}{x_i!} = \left(\prod_{i=1}^n \frac{1}{x_i!}\right) e^{-n\theta}\theta^{\sum_{i=1}^n X_i}$$

- Thus $f(X|\theta)$ has been factorized as the product of a function that does not depend on $\theta$ and a function that depends on $\theta$ but depends on the observed vector $X$ only through the value of $\sum_{i=1}^n X_i$. By the factorization theorem, it follows that $T = \sum_{i=1}^n X_i$ is a sufficient statistic for $\theta$.

## Sufficient Statistics and Estimators

- We now know that estimators are statistics, in particular, we want the obtained estimator to be a sufficient statistic, since we want an estimator that absorbs all the available information contained in the sample.
- Suppose that $X_1, \ldots, X_n$ form a random sample from the distribution for which the pdf or point mass function is $f(x|\theta)$, where the value of the parameter $\theta$ is unknown.
- We assume that there is a sufficient statistic of $\theta$, which is $T(X_1, \ldots, X_n)$.
- We can show that the MLE of $\theta$, $\hat{\theta}$, depends on $X_1, \ldots, X_n$ only through the statistic $T$.
- From the factorization theorem, it follows that the likelihood function $f_n(X|\theta)$ can be written as:

$$f_n(X|\theta) = u(X)v[T(X), \theta]$$

- We know that the MLE $\hat{\theta}$ is the value of $\theta$ for which $f_n(X|\theta)$ is maximized. We also know that u and v and positive.
- Therefore, it follows that $\hat{\theta}$ will be the value of $\theta$ for which $v[T(X), \theta]$ is maximized. Since $v[T(X), \theta]$ depends on $X$ only through the function $T(X)$, it follows that $\hat{\theta}$ will depend on $X$ only through the function $T(X)$.
- Thus the MLE estimator $\hat{\theta}$ is a function of the sufficient statistic $T(X_1, \ldots, X_n)$.
- In many statistical problems, the MLE $\hat{\theta}$ is actually a sufficient statistic.

## Exponential family of probability distributions

- Many common distributions eg normal, binomial, Poisson, Gamma, etc are members of the exponential family of distributions.
- One parameter members of the exponential family distributions have density or mass function of the form:

$$f(x|\theta) = exp\left[c(\theta)T(X) + d(\theta) + S(X)\right]$$

- Suppose that $X_1 \ldots, X_n$ are iid samples from a member of the exponential family, then the joint probability function is:

$$f(X|\theta) = \prod_{i=1}^{n} exp\left[c(\theta)T(X) + d(\theta) + S(X)\right] = exp\left[c(\theta)\sum_{i=1}^{n} T(X) + nd(\theta)\right] exp\left[\sum_{i=1}^{n} S(X)\right]$$

- From the Factorization Theorem, it is apparent that $\sum_{i=1}^{n} T(X_i)$ is a sufficient statistic.

- Example: Let $X_1, \ldots, X_n$ be iid from a Bernoulli distribution with mass function:

$$P(X = x) = \theta^x (1-\theta)^{1-x}, \qquad x = 0, 1$$

- Rewriting in exponential form

$$exp\left[xlog\ \theta + (1-x)log(1-\theta)\right] = exp\left[xlog\ \theta + log(1-\theta) - xlog(1-\theta)\right]$$

$$= exp\left[xlog\ \left(\frac{\theta}{1-\theta}\right) + log(1-\theta)\right]$$

- Thus this is a member of the exponential family with $T(X) = X$, thus $\sum_{i=1}^{n} X_i$ is a sufficient statistic.

## Example: MLE for binary outcome using logistic regression

Consider a bag with only two types of balls: White and red. Let $p$ be the fraction of the white balls in the bag. The value of $p$ is unknown. Suppose that 100 balls are drawn from the bag and 30 are white. What is the best estimate of $p$?

In the context of MLE, $p$ is the parameter in the model we are trying to estimate. Question is: given $p$, what is the probability that we get 30 white balls from 100 draws? The probability that we get a white ball in each draw is $p$, and the probability that we get a red ball is $1-p$, so the probability of getting 30 white balls and 70 red balls in 100 draws is:

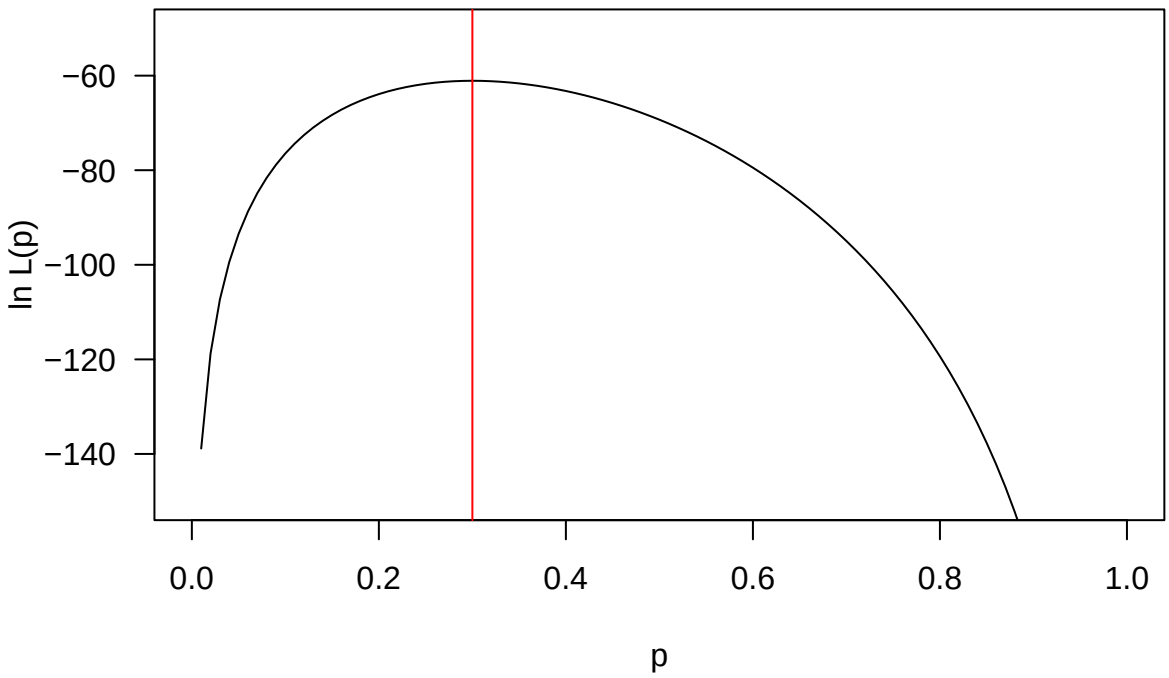
$$L(p) = p(30|p) = p^{30}(1-p)^{70}$$

This is a function of the unknown parameter $p$, called the likelihood function. It is the probability of getting the data if the parameter is equal to $p$. The maximum likelihood estimate for the parameter is the value of $p$ that maximizes the likelihood function. Instead of working with the likelihood function $L(p)$, it is more convenient to work with the logarithm of $L(p)$:

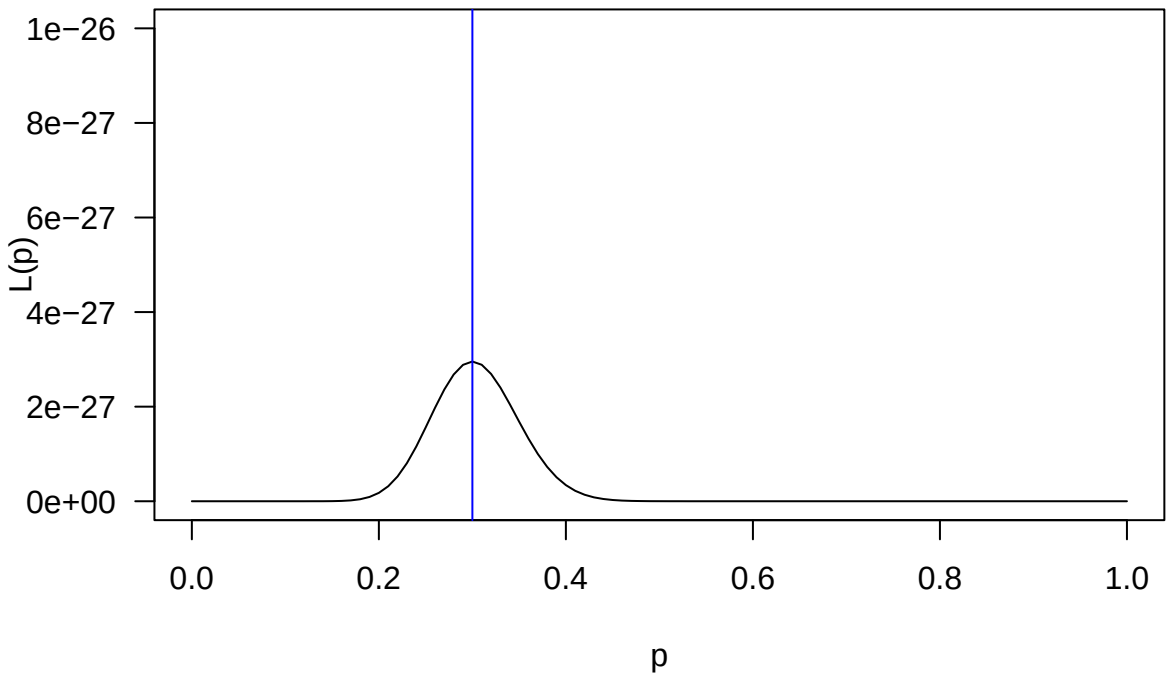
$$lnL(p) = 30ln(p) + 70ln(1-p)$$

NB: since $ln(x);\ \forall\ x > 0$, is an increasing function of $x$, maximizing $L(p)$ is the same as maximizing $lnL(p)$.The MLE of $p$ then is obtained by finding the $p$ that maximizes the function $lnL(p) = 30ln(p) + 70ln(1-p)$. This can be visualized:

```
#---------------------------------------
# Likelihood for binomial example
#--------------------------------------

curve(30 * log(x) + 70 * log(1 - x), xlim = c(0, 1), ylim = c(-150,
    -50), las = 1, xlab = "p", ylab = "ln L(p)")
abline(v = 0.3, col = "red")
```




```r
curve(x^30 * (1 - x)^70, xlim = c(0, 1), ylim = c(0, 1e-26),
    las = 1, xlab = "p", ylab = "L(p)")
abline(v = 0.3, col = "blue")
```




This likelihood can be expressed in terms of $y_i$. If $y_i = 1$ we get the white ball and if $y_i = 0$ we have the red ball. The likelihood function is the probability that we get $y_1, y_2, \ldots, y_n$ from N draws:

$$L(p) = p(y_1, y_2, \ldots, y_n|p) = p^{y_1}(1-p)^{1-y_1} \times p^{y_2}(1-p)^{1-y_2} \cdots \times p^{y_n}(1-p)^{1-y_n}$$
$$= \Pi_{i=1}^{n} p^{y_i}(1-p)^{1-y_i}$$

with the log-likelihood given by

$$lnL(p) = \sum_{i=1}^{N} [y_i ln(p) + (1-y_i)(1-p)]$$

and $p = \frac{a}{N} = \bar{y}$; where $a = \sum_i y_i$

### Credit card defaults (real data example MLE: logistic regression)

We are going to use the credit card defaulting dataset which you can access through the provided link.Explore and manage the data.

```
Default <- read.csv("http://courses.atlas.illinois.edu/spring2016/STAT/STAT200/RProgramming/data/Default

head(Default, 5)
```

```
##   default student balance income
## 1      No      No   729.5  44362
## 2      No     Yes   817.2  12106
```

```
## 3      No      No  1073.5  31767
## 4      No      No   529.3  35704
## 5      No      No   785.7  38463
```

```r
# create y as a vector
y <- as.integer(Default$default == "Yes")

# MLE
mean(y)
```

```
## [1] 0.0333
```

```r
# about 3.3% defaults

tapply(y, Default$student, mean)
```

```
##      No     Yes
## 0.02920 0.04314
```

```r
# 4% student defaults and non-students 3%

# explore the balance variable
summary(Default$balance)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       0     482     824     835    1166    2654
```

```r
# create a factor variable that splits splits the balance
# variable into 10
balance_cut1 <- cut(Default$balance, breaks = 10)
levels(balance_cut1)
```

```
##  [1] "(-2.65,265]"         "(265,531]"           "(531,796]"
##  [4] "(796,1.06e+03]"      "(1.06e+03,1.33e+03]" "(1.33e+03,1.59e+03]"
##  [7] "(1.59e+03,1.86e+03]" "(1.86e+03,2.12e+03]" "(2.12e+03,2.39e+03]"
## [10] "(2.39e+03,2.66e+03]"
```

```r
# see number of observations in each category
table(balance_cut1)
```

```
## balance_cut1
##         (-2.65,265]           (265,531]           (531,796]
##                1315                1501                1975
##      (796,1.06e+03] (1.06e+03,1.33e+03] (1.33e+03,1.59e+03]
##                2044                1568                 941
## (1.59e+03,1.86e+03] (1.86e+03,2.12e+03] (2.12e+03,2.39e+03]
##                 447                 157                  44
## (2.39e+03,2.66e+03]
##                   8
```

```r
# can also do quantiles
quantiles <- quantile(Default$balance, probs = seq(0, 1, length = 11))
quantiles
```

```
##     0%    10%    20%    30%    40%    50%    60%    70%    80%    90%
##    0.0  180.6  395.4  561.6  697.2  823.6  951.3 1086.2 1249.8 1471.6
##   100%
## 2654.3
```

```r
# use quantiles and cut functions'
balance_cut2 = cut(Default$balance, breaks = quantiles)
levels(balance_cut2)
```

```
##  [1] "(0,181]"             "(181,395]"           "(395,562]"
##  [4] "(562,697]"           "(697,824]"           "(824,951]"
##  [7] "(951,1.09e+03]"      "(1.09e+03,1.25e+03]" "(1.25e+03,1.47e+03]"
## [10] "(1.47e+03,2.65e+03]"
```

```r
table(balance_cut2)
```

```
## balance_cut2
##             (0,181]           (181,395]           (395,562]
##                 501                1000                1000
##           (562,697]           (697,824]           (824,951]
##                1000                1000                1000
##      (951,1.09e+03] (1.09e+03,1.25e+03] (1.25e+03,1.47e+03]
##                1000                1000                1000
## (1.47e+03,2.65e+03]
##                1000
```

```r
# to include '0'
balance_cut2 = cut(Default$balance, breaks = quantiles, include.lowest = TRUE)
table(balance_cut2)
```

```
## balance_cut2
##             [0,181]           (181,395]           (395,562]
##                1000                1000                1000
##           (562,697]           (697,824]           (824,951]
##                1000                1000                1000
##      (951,1.09e+03] (1.09e+03,1.25e+03] (1.25e+03,1.47e+03]
##                1000                1000                1000
## (1.47e+03,2.65e+03]
##                1000
```

```r
# proportion of defaults in each quantile
p <- tapply(y, balance_cut2, mean)
p
```

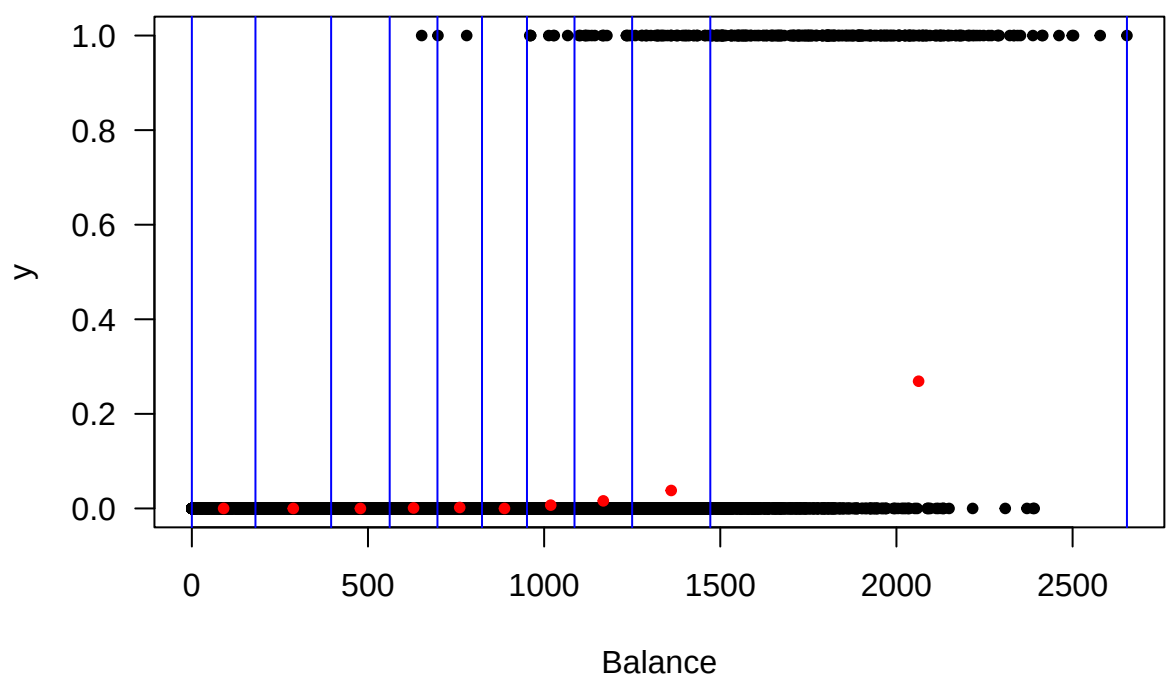
```
##             [0,181]           (181,395]           (395,562]
##               0.000               0.000               0.000
```

```
##           (562,697]           (697,824]           (824,951]
##               0.001               0.002               0.000
##      (951,1.09e+03] (1.09e+03,1.25e+03] (1.25e+03,1.47e+03]
##               0.007               0.016               0.038
## (1.47e+03,2.65e+03]
##               0.269
```

```r
# visualize the result
balance_cut2_midpoints = rep(NA, 10)  # initialize a vector of length 10
for (i in 1:10) {
    balance_cut2_midpoints[i] = (quantiles[i] + quantiles[i +
        1])/2
}
balance_cut2_midpoints
```

```
##  [1]   90.29  287.98  478.50  629.39  760.40  887.44 1018.72 1167.99
##  [9] 1360.71 2062.97
```

```r
plot(y ~ Default$balance, las = 1, pch = 20, xlab = "Balance")
points(balance_cut2_midpoints, p, col = "red", pch = 20)
abline(v = quantiles, col = "blue")
```




```r
# The red points are the maximum likelihood estimate for
# the fractions of defaulted customers in the intervals.
```

```r
# split last interval
quan_last <- quantile(Default$balance, probs = c(0.92, 0.94,
    0.96, 0.98, 1))

# combine with other quantiles
quan_combined <- c(quantiles[1:10], quan_last)
quan_combined
```

```
##     0%    10%    20%    30%    40%    50%    60%    70%    80%    90%
##    0.0  180.6  395.4  561.6  697.2  823.6  951.3 1086.2 1249.8 1471.6
##    92%    94%    96%    98%   100%
## 1534.7 1615.2 1724.4 1867.3 2654.3
```

```r
# construct 14 categories
balance_cut3 <- cut(Default$balance, breaks = quan_combined,
    include.lowest = TRUE)
table(balance_cut3)
```

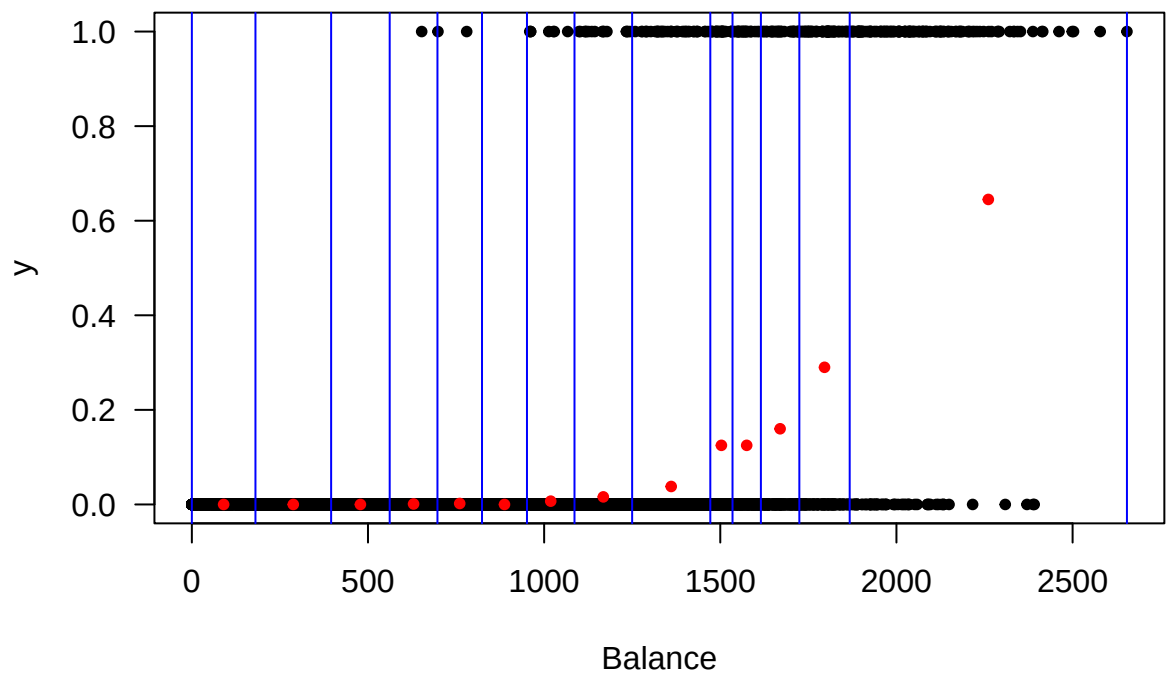
```
## balance_cut3
##             [0,181]           (181,395]           (395,562]
##                1000                1000                1000
##           (562,697]           (697,824]           (824,951]
##                1000                1000                1000
##      (951,1.09e+03] (1.09e+03,1.25e+03] (1.25e+03,1.47e+03]
##                1000                1000                1000
## (1.47e+03,1.53e+03] (1.53e+03,1.62e+03] (1.62e+03,1.72e+03]
##                 200                 200                 200
## (1.72e+03,1.87e+03] (1.87e+03,2.65e+03]
##                 200                 200
```

```r
# the mle for each block
p <- tapply(y, balance_cut3, mean)
p
```

```
##             [0,181]           (181,395]           (395,562]
##               0.000               0.000               0.000
##           (562,697]           (697,824]           (824,951]
##               0.001               0.002               0.000
##      (951,1.09e+03] (1.09e+03,1.25e+03] (1.25e+03,1.47e+03]
##               0.007               0.016               0.038
## (1.47e+03,1.53e+03] (1.53e+03,1.62e+03] (1.62e+03,1.72e+03]
##               0.125               0.125               0.160
## (1.72e+03,1.87e+03] (1.87e+03,2.65e+03]
##               0.290               0.645
```

```r
# This show how the fraction of defaults changes in the
# last few intervals. We can make another plot summarizing
# the result.
nintervals = length(levels(balance_cut3))  # Number of intervals
balance_cut3_midpoints <- rep(NA, nintervals)  # initialize a vector of length nintervals
# Compute the midpoints of intervals
```

```r
for (i in 1:nintervals) {
    balance_cut3_midpoints[i] <- (quan_combined[i] + quan_combined[i +
        1])/2
}
# Plot
plot(y ~ Default$balance, las = 1, pch = 20, xlab = "Balance")
points(balance_cut3_midpoints, p, col = "red", pch = 20)
abline(v = quan_combined, col = "blue")
```




The curve represents a logistic curve. In MLE, we want to maximize the log-likelihood function:

$$lnL(\{p(x)\}) = \sum_{i=1}^{n} \{y_i lnp(x_i) + (1 - y_i) ln[1 - p(x_i)]\}$$

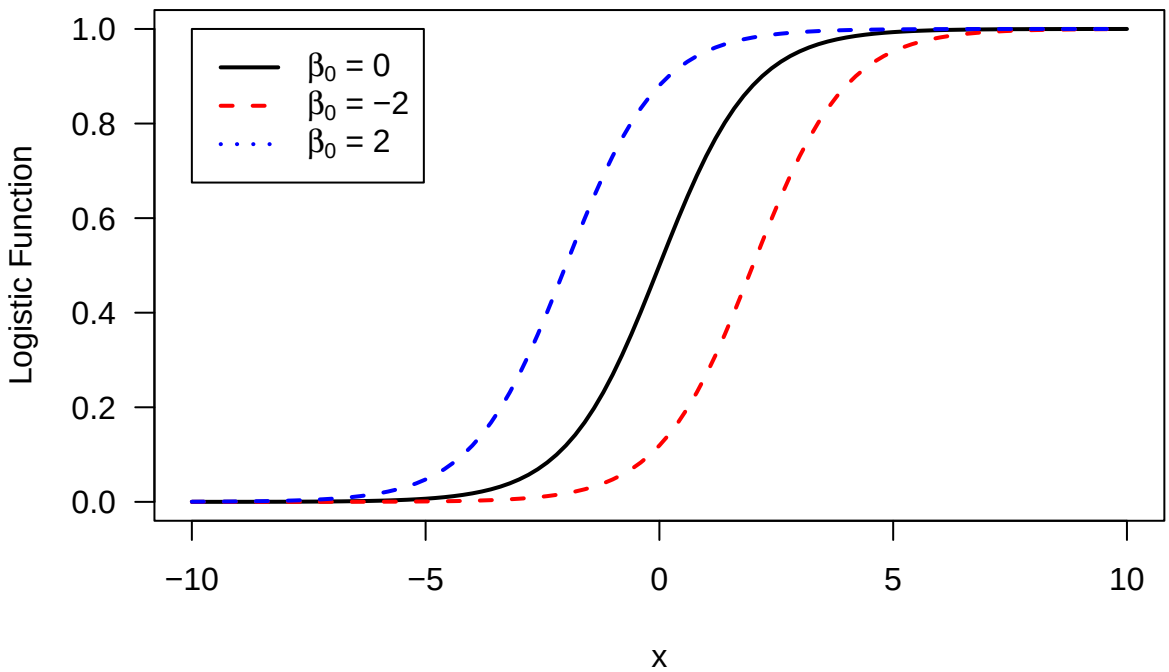
If $x$ is a factor variable with $k$ levels, $\{p(x_i)\}$ contains $k$ values corresponding to the $k$ parameters $p_1, p_2, \ldots, p_n$. If $x$ is a continuous variable, then $\{p(x_i)\}$ in principle has an infinite values. In a one variable logistic regression, we specify the function as:

$$p(x) = p(x_i; \beta_0, \beta_1) = \frac{e^{\beta_0 + \beta_1 x}}{1 + e^{\beta_0 + \beta_1 x}} = \frac{1}{1 + e^{-\beta_0 - \beta_1 x}}$$

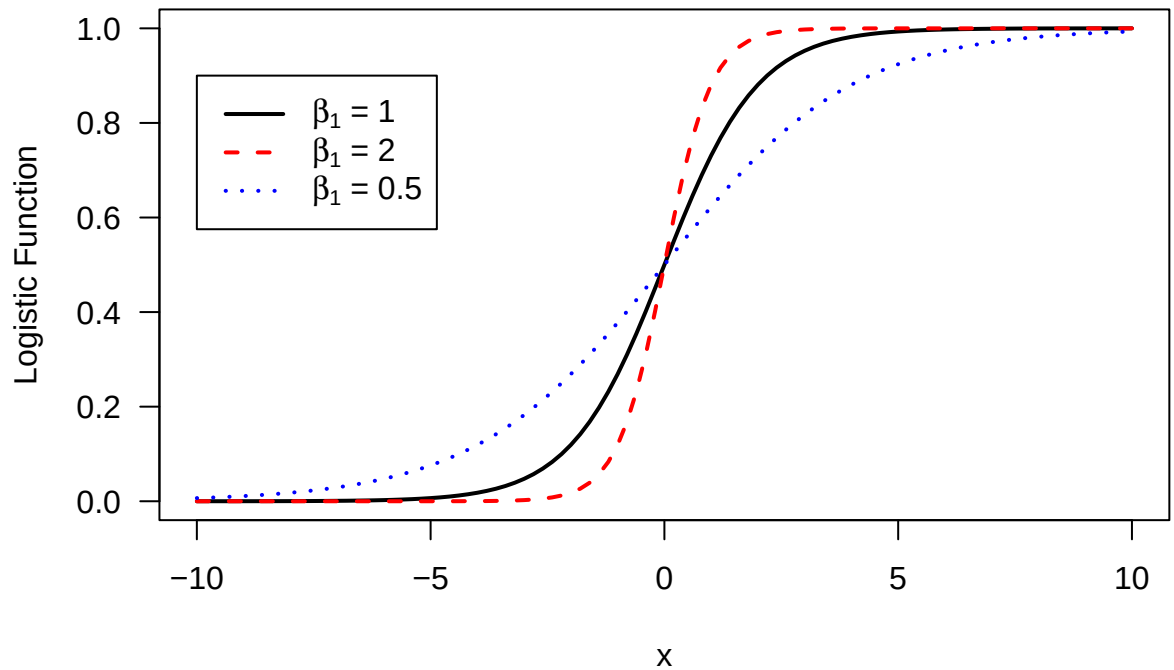
This is called the logistic function or the sigmoid function. The curve is S-shaped.

```r
# logistic curves Define the logistic function
logit <- function(x, beta0, beta1) {
    1/(1 + exp(-beta0 - beta1 * x))
```

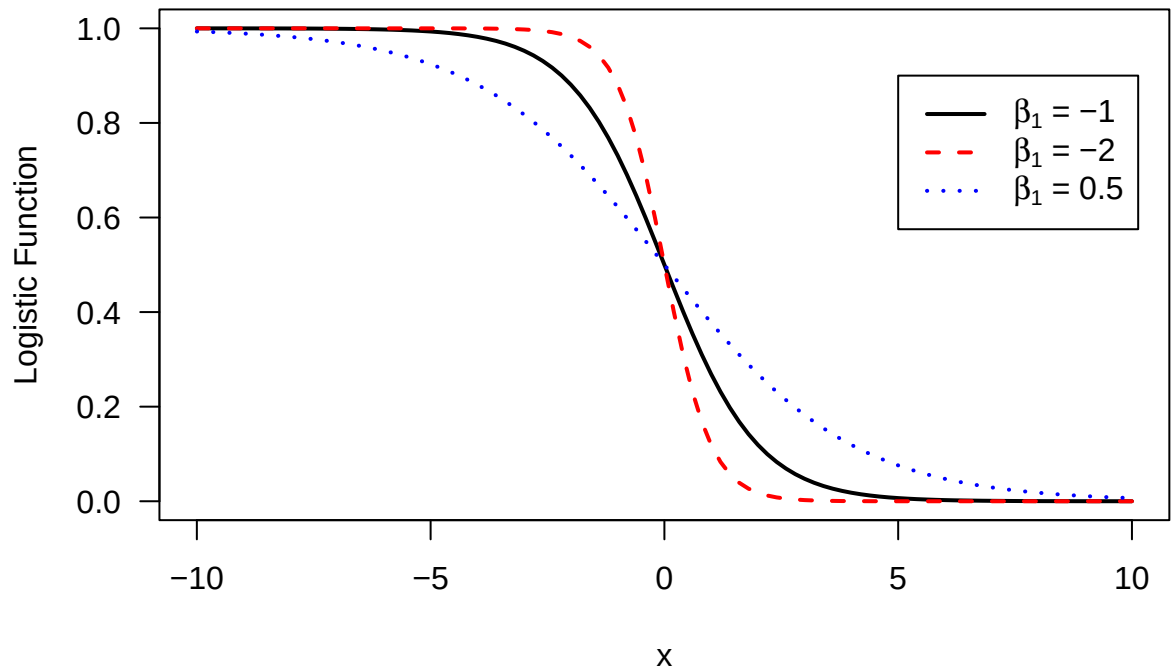
```r
}

# Plot the logistic function with beta1=1 and 3 different
# values of beta0.
curve(logit(x, 0, 1), xlim = c(-10, 10), lwd = 2, ylab = "Logistic Function",
    las = 1)
curve(logit(x, -2, 1), xlim = c(-10, 10), lwd = 2, lty = 2, col = "red",
    add = TRUE)
curve(logit(x, 2, 1), xlim = c(-10, 10), lwd = 2, lty = 2, col = "blue",
    add = TRUE)
legend(-10, 1, c(expression(paste(beta[0], " = 0")), expression(paste(beta[0],
    " = -2")), expression(paste(beta[0], " = 2"))), lwd = 2,
    lty = 1:3, col = c("black", "red", "blue"))
```




```r
# Plot the logistic function with beta0=0 and 3 different
# +ve values of beta1.
curve(logit(x, 0, 1), xlim = c(-10, 10), lwd = 2, ylab = "Logistic Function",
    las = 1)
curve(logit(x, 0, 2), xlim = c(-10, 10), lwd = 2, lty = 2, col = "red",
    add = TRUE)
curve(logit(x, 0, 0.5), xlim = c(-10, 10), lwd = 2, lty = 3,
    col = "blue", add = TRUE)
legend(-10, 0.9, c(expression(paste(beta[1], " = 1")), expression(paste(beta[1],
    " = 2")), expression(paste(beta[1], " = 0.5"))), lwd = 2,
    lty = 1:3, col = c("black", "red", "blue"))
```




```r
# Plot the logistic function with beta0=0 and 3 different
# -ve values of beta1.
curve(logit(x, 0, -1), xlim = c(-10, 10), lwd = 2, ylab = "Logistic Function",
    las = 1)
curve(logit(x, 0, -2), xlim = c(-10, 10), lwd = 2, lty = 2, col = "red",
    add = TRUE)
curve(logit(x, 0, -0.5), xlim = c(-10, 10), lwd = 2, lty = 3,
    col = "blue", add = TRUE)
legend(5, 0.9, c(expression(paste(beta[1], " = -1")), expression(paste(beta[1],
    " = -2")), expression(paste(beta[1], " = 0.5"))), lwd = 2,
    lty = 1:3, col = c("black", "red", "blue"))
```




```r
# b1 values detemine how steep the curve is for both +ve
# and -ve values while b0 shift curve from left to right
```

The logistic function is constructed such that the log-odds, $log\left[\frac{p}{(1-p)}\right]$ is a linear function of $x$. There are two parameters $\beta_0$ *and* $\beta_1$ in the function and they are determined by maximizing the Log-Likelihood function:

$$lnL(\beta_0, \beta_1) = \sum_{i=1}^{n} \{y_i lnp(x_i; \beta_0, \beta_1) + (1 - y_i) ln[1 - p(x_i; \beta_0, \beta_1)]\}$$

Maximization for this function uses numerical methods since nonlinear and analytically intractable! In R we use the *glm* ie generalized linear models framework!

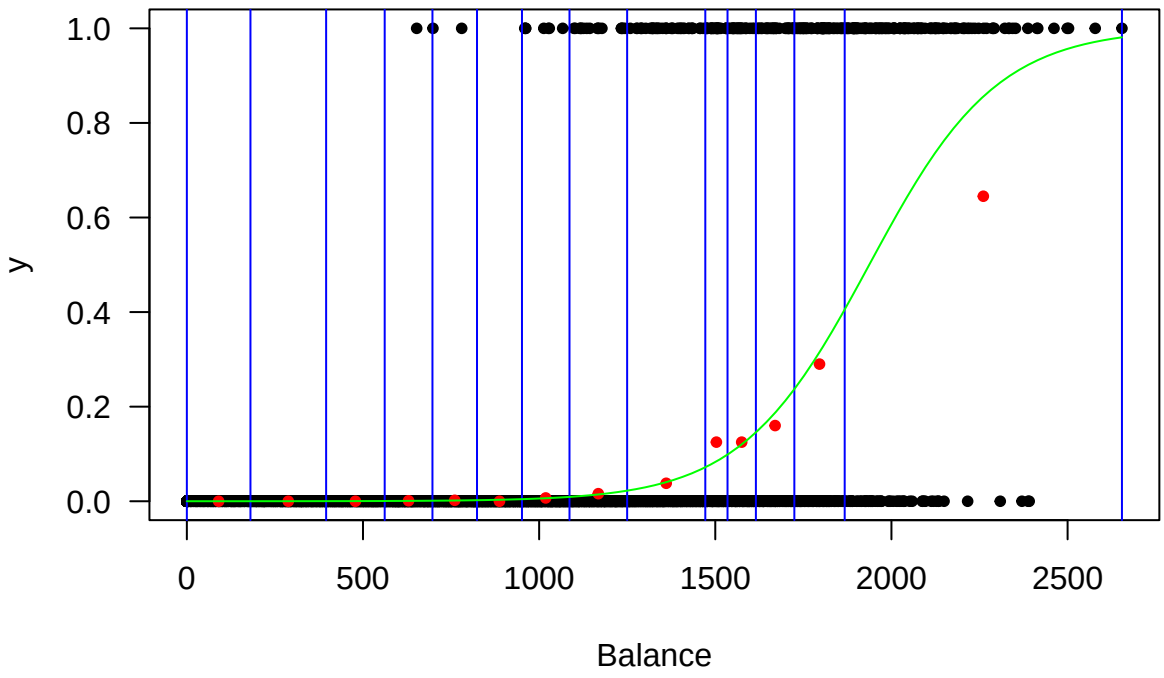
```r
# mle estimates for betas
fit <- glm(y ~ Default$balance, family = binomial)
fit
```

```
##
## Call:  glm(formula = y ~ Default$balance, family = binomial)
##
## Coefficients:
##     (Intercept)  Default$balance
##        -10.6513           0.0055
##
## Degrees of Freedom: 9999 Total (i.e. Null);  9998 Residual
## Null Deviance:       2920
## Residual Deviance: 1600  AIC: 1600
```

```r
summary(fit)
```

```
##
## Call:
## glm(formula = y ~ Default$balance, family = binomial)
##
## Coefficients:
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -10.65133    0.36116   -29.5   <2e-16 ***
## Default$balance   0.00550    0.00022    24.9   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 2920.6  on 9999  degrees of freedom
## Residual deviance: 1596.5  on 9998  degrees of freedom
## AIC: 1600
##
## Number of Fisher Scoring iterations: 8
```

```r
# We can add the logistic curve with these values of b0 and
# b1 to the plot of y vs 'balance' above:

# Re-create the y vs 'balance' plot
plot(y ~ Default$balance, las = 1, pch = 20, xlab = "Balance")
points(balance_cut3_midpoints, p, col = "red", pch = 20)
abline(v = quan_combined, col = "blue")
# Add a logistic curve with beta0 = fit$coefficients[1],
# beta1 = fit$coefficients[2]
curve(logit(x, fit$coefficients[1], fit$coefficients[2]), col = "green",
    add = TRUE)
```


1.0
0.8
0.6
0.4
0.2
0.0
y
0
500
1000
1500
2000
2500
Balance